

Trabalho de Conclusão de Curso

Árvores de comportamento: Uma abordagem para sistemas embarcados

José Renilson Almeida da Silva

Orientador:

Prof. Me. Rodrigo José Sarmento Peixoto

Maceió, Junho de 2025

José Renilson Almeida da Silva

Árvores de comportamento: Uma abordagem para sistemas embarcados

Monografia apresentada como requisito parcial para obtenção do grau de Bacharel em Engenharia de Computação do Instituto de Computação da Universidade Federal de Alagoas.

Orientador:

Prof. Me. Rodrigo José Sarmento Peixoto

Maceió, Junho de 2025

Catalogação na Fonte
Universidade Federal de Alagoas
Biblioteca Central
Divisão de Tratamento Técnico
Bibliotecário: Marcelino de Carvalho Freitas Neto – CRB-4 – 1767

S586a Silva, José Renilson Almeida da.
Árvores de comportamento : uma abordagem para sistemas embarcados / José Renilson Almeida da Silva. – Maceió, 2025.
61 f. : il., grafs. e tabs. color.

Orientador: Rodrigo José Sarmento Peixoto.
Monografia (Trabalho de conclusão de curso em Engenharia de Computação) – Universidade Federal de Alagoas. Instituto de Computação. Maceió, 2025.

Bibliografia: f. 59-60.
Apêndices: f. 61.

1. Árvores de comportamento. 2. Sistemas embarcados (Computadores). 3. Grafo de comportamento mínimo. I. Título.

CDU: 004.422.635.3

UNIVERSIDADE FEDERAL DE ALAGOAS

Instituto de Computação

Árvores de comportamento: Uma abordagem para sistemas embarcados

Trabalho de Conclusão de Curso apresentado ao Instituto de Computação da Universidade Federal de Alagoas como requisito parcial para a obtenção do grau de Bacharel em Engenharia de Computação, aprovada pela comissão examinadora que abaixo assina:

Documento assinado digitalmente
gov.br
RODRIGO JOSE SARMENTO PEIXOTO
Data: 30/07/2025 15:20:56-0300
Verifique em https://validar.iti.gov.br

Prof. Me. Rodrigo José Sarmento Peixoto - Orientador

Instituto de Computação

Universidade Federal de Alagoas

ERICK DE ANDRADE BARBOZA:07171132480
Assinado de forma digital por ERICK DE ANDRADE BARBOZA:07171132480
Dados: 2025.06.25 10:27:30 -03'00'

Prof. Dr. Erick de Andrade Barboza - Examinador

Instituto de Computação

Universidade Federal de Alagoas

Documento assinado digitalmente
gov.br
ALFREDO LIMA MOURA SILVA
Data: 23/06/2025 12:16:08-0300
Verifique em https://validar.iti.gov.br

Me. Alfredo Lima Moura Silva - Examinador

Instituto de Computação

Universidade Federal de Fluminense

Documento assinado digitalmente
gov.br
OTAVIO REAL SALVADOR
Data: 03/07/2025 11:05:03-0300
Verifique em https://validar.iti.gov.br

Otávio Real Salvador - Examinador

CEO

O.S. Systems

Agradecimentos

Agradeço primeiramente à minha família, pelo apoio incondicional em todos os momentos desta jornada — em especial à minha mãe, Rose Mary, e aos meus irmãos, Aldemir Almeida e Izaiane Almeida. Agradeço também à minha parceira, Taiane Alves, pelo suporte constante e pela presença fundamental durante essa fase.

Gostaria de agradecer especialmente ao Alfredo Lima, por me apresentar conceitos e ideias que contribuíram significativamente para a construção deste trabalho. Agradeço também a Jeferson Fernando, Victor Oliveira e Matheus Tenório pelas discussões valiosas sobre o tema.

Agradeço ainda ao meu orientador, Rodrigo Peixoto, pelo incentivo e orientação ao longo do desenvolvimento deste trabalho, assim como a todos os colegas do laboratório EDGE-Embarcados.

Por fim, agradeço aos professores do Instituto de Computação da UFAL, pela excelência no ensino e dedicação ao longo da minha formação.

Resumo

Sistemas embarcados exigem soluções capazes de lidar com tarefas concorrentes e normalmente repetitivas, mantendo ao mesmo tempo modularidade, clareza e baixo consumo de recursos. Modelar soluções de forma escalável e reutilizável continua sendo um desafio recorrente no desenvolvimento embarcado. Este trabalho propõe uma abordagem estruturada para organização lógica de componentes da aplicação de comportamentos. A solução é baseada em Árvores de Comportamento, uma estrutura hierárquica originalmente usada em jogos e robótica que permite descrever agentes autônomos de forma modular e flexível, adaptada aqui para sistemas embarcados. A solução desenvolvida visa implementar o comportamento previsto da árvore, substituindo o modelo convêncional topológico da árvore, de contexto por uma estrutura mais leve e direta. Os resultados demonstram que a solução embarcada tem potêncial, independente do *Real-Time Operational System* (*RTOS*) que seja usado. Além disso, a abordagem proposta permite integração ou substituição de estruturas tradicionais, como as FSMs, e possibilita a manipulação e orquestração de múltiplas threads, com várias árvores em execução paralela.

Palavras-chave: Árvore de Comportamento; Estrutura; Sistema Embarcado; Grafo de Comportamento Mínimo.

Abstract

Embedded systems require solutions capable of handling concurrent and often repetitive tasks, while maintaining modularity, clarity, and low resource consumption. Designing scalable and reusable solutions remains a recurring challenge in embedded development. This work proposes a structured approach for the logical organization of application behavior components. The solution is based on Behavior Trees, a hierarchical structure originally used in games and robotics to describe autonomous agents in a modular and flexible manner, adapted here for embedded systems. The developed solution aims to implement the expected behavior of the tree by replacing the conventional context-based topological model with a lighter and more direct structure. The results show that the embedded solution has potential regardless of the Real-Time Operating System (RTOS) in use. Furthermore, the proposed approach enables the integration or replacement of traditional structures such as FSMs, and allows the management and orchestration of multiple threads, with several trees running in parallel.

Keywords: *Behavior Tree*; *Structure*; *Embedded System*; *Minimum Behavior Graph*.

Lista de Figuras

1 Grafo cíclico. Fonte: Elaborado pelo autor, (2025) 4
2 Grafo acíclico. Fonte: Elaborado pelo autor, (2025) 4
3 FSM antes de modificação. Fonte: Elaborado pelo autor, (2025) 7
4 FSM com a implementação do nó *D* adicional. Fonte: Elaborado pelo autor, (2025) 7
5 Árvore de comportamento simples. Fonte: Elaborado pelo autor, (2025) 13
6 Ramificação repetitiva. Fonte: Elaborado pelo autor, (2025) 15
7 Extensão da ramificação. Fonte: Elaborado pelo autor, (2025) 15
8 Execução convencional de uma árvore de comportamento 19
9 Estrutura com a abordagem de referências dos irmãos. 20
10 Execução simplificada da estrutura. 22
11 Execução de uma ramificação com grafo de comportamento mínimo. 24
12 Troca de ramificações de árvore de comportamento. 25
13 Troca de ramificações no grafo de comportamento mínimo. 26
14 Nó MoverParaB em Execução. 28
15 Tick no nó Reactive Sequence a esquerda. 28
16 A condição *InimigoProximo?* é satisfeita e então a ramificação é continuada. . 28
17 *DevKit EFR32xG24-DK2601B*. 30
18 *DevKit E-Paper ESP32* Driver Board. 30
19 Array intermediário de criação. 35
20 Árvore reativa de exemplo. 38
21 Array da estrutura mínima da árvore de comportamento Exemplo. 39
22 Log da estrutura mínima reativa Exemplo. 39
23 Saída da árvore de comportamento executada pela biblioteca BehaviorTree.CPP. 40
24 Saída da estrutura mínima sem reatividade. 40
25 Árvore de comportamento simples sendo executada pelo biblioteca Behavior-Tree.CPP, log 1. 41
26 Árvore de comportamento simples sendo executada pelo biblioteca Behavior-Tree.CPP, log 2. 41
27 Uma das ramificações da árvore de comportamento *MSGS*. 42
28 Subárvore *process*. 42
29 Saída da estrutura mínima sem reatividade, log 1. 44

30 Saída da estrutura mínima sem reatividade, log 2. 44
31 Saída da estrutura mínima sem reatividade, log 3. 44
32 Árvore reativa da SubTree. 45
33 Array da estrutura mínima da árvore de comportamento SubTree. 46
34 Log da estrutura mínima reativa SubTree. 46
35 Logs das estruturas mínima reativa de Exemplo e SubTree. 48
36 Árvore principal da aplicação bluetooth com Wi-Fi. 49
37 Subárvore bluetooth. 49
38 Subárvore Wi-Fi. 50
39 Array da estrutura Bluetooth e Wi-Fi. 51
40 Primeiro log da aplicação BLE + Wi-Fi. 51
41 Segundo log da aplicação BLE + Wi-Fi. 52
42 Terceiro *log* da aplicação *BLE + Wi-Fi*. 53
43 Quarto log da aplicação *BLE + Wi-Fi*. 53
44 Árvore de comportamento para mensagem *bluetooth*. 54
45 Estrutura mínima de comportamento para mensagem *bluetooth*. 55
46 Log 1 da estrutura mínima de comportamento para mensagem bluetooth 55
47 Log 2 da estrutura mínima de comportamento para mensagem bluetooth. 55
48 Log 3 da estrutura mínima de comportamento para mensagem bluetooth. 55
49 *Log* não reativo da estrutura mínima de comportamento para mensagem *bluetooth*. 56

Lista de Tabelas

4.1 Tabela de seções de memória e seus tamanhos. 37
4.2 Comparação de chamadas de nós 47

Conteúdo

1 Introdução **1**

1.1 Introdução 1

1.2 Objetivos 2

1.2.1 Objetivos específicos 2

2 Fundamentação Teórica **3**

2.1 Grafos 3

2.2 Árvores 5

2.2.1 Uma necessidade comportamental 6

2.3 Árvore de comportamento 8

2.3.1 Nós 8

2.3.2 Tick 12

2.3.3 Estruturação 12

2.3.4 Blackboard 14

2.4 Complexidade da Árvore de comportamento 14

2.5 Modelagem de árvores de comportamento 15

3 Metodologia **17**

3.1 Recurso embarcado 17

3.2 Desafios de uma árvore de comportamento embarcada 18

3.3 Proposta prática: Otimização utilizando Pilha e Modelagem K-ária 20

3.4 Proposta prática: Grafo simplificado da estrutura sem pilha 21

3.5 Grafo de comportamento mínimo 23

3.5.1 Princípios da conversão da Árvore de Comportamento para Grafo de comportamento mínimo 27

3.6 Reatividade 28

3.7 Testes e Validação 29

4 Resultados e discussões **31**

4.1 Suporte embarcado 31

4.2 Interações 32

4.3 Blackboard 32
4.4 Estados 33
4.5 Scripts de Pré-processamento 34
4.6 Processamento 35
4.7 Gerenciamento 36
4.8 Consumo 36
4.9 Resultados 37
4.9.1 Primeiro caso: Teste de reatividade 37
4.9.2 Segundo caso: Teste de mensagem 42
4.9.3 Terceiro caso: Teste de reatividade com decorators 45
4.9.4 Quarto caso: Teste em conjunto 48
4.9.5 Quinto caso: Teste de aplicação Bluetooth e Wi-Fi 49
4.9.6 Sexto caso: Teste de aplicação *bluetooth* com mensagens 54

5 Conclusão **57**
5.1 Trabalhos Futuros 58

Referências **60**

A Repositório de Código-Fonte **61**

1

Introdução

1.1 Introdução

As tecnologias embarcadas fazem parte do cotidiano e em uma ampla variedade de dispositivos e aplicações. Semáforos inteligentes, relógios digitais, telefones celulares, rastreadores veiculares, controladores de lâmpadas, sistemas de climatização e muitos outros dispositivos utilizam sistemas embarcados para executar tarefas específicas com eficiência e confiabilidade. Segundo White (2024), um sistema embarcado é tudo aquilo que executam softwares, mas que não é um computador, sendo um sistema computadorizado especificado para a sua aplicação.

Tecnologias embarcadas normalmente possuem restrições de memória, consumo de energia, custo ou recursos disponíveis, o que influencia as escolhas de arquitetura e a forma de organizar a lógica da aplicação. Ferramentas consolidadas, por exemplo Máquinas de Estado Finito (FSM - *Finite State Machine*), que ajudam a projetar sistemas, oferecendo uma implementação genérica e abrangente ao projeto, sendo bem estruturadas e com uma base sólida.

Embora as FSMs se adequem a diversos projetos, à medida que novas funcionalidades forem incorporadas, a manutenção e a modularidade se tornam mais difíceis (COLLEDANCHISE; ÖGREN, 2018). A adição de um estado pode provocar um aumento exponencial na complexidade, devido ao crescimento das possíveis transições entre eventos ou estados. Além disso, quanto mais reativo for um sistema, maior será o número de transições realizadas, o que pode ser problemático, uma vez que cada transição em uma FSM corresponde ao tratamento de um comando *goto*.

Uma estrutura desenvolvida por volta de 2004 é a árvore de comportamento (*Behavior Tree*), inicialmente proposta como uma alternativa às FSMs em áreas como jogos digitais e robótica. As árvores de comportamento foram desenvolvidas para lidar com limitações das FSMs em cenários que exigem comportamentos complexos, oferecendo modularidade e reatividade. Sua arquitetura hierárquica permite organizar decisões de forma clara e escalável, deixando a estrutura com um grau de complexidade reduzida, onde a transição entre seus comportamentos tem

como base seus nós intermediários e da esquerda para a direita, podendo substituir totalmente as FSMs.

A complexidade inerente a sistemas embarcados e suas limitações em questão de recursos computacionais, exigem soluções modernas, flexíveis e robustas para modelagem de comportamentos. Em aplicações que demandam comportamentos previsíveis, sequenciais ou hierárquicos, uma FSM pode apresentar limitações principalmente no aspecto de manutenção. Diante disso, o estudo de uma solução alternativa para a modelagem de lógicas de controle e a implementação de estruturas hierárquicas, como uma árvore de comportamento, se torna relevante por oferecer uma modularidade, abstração e simplicidade de implementação.

A proposta não visa realizar uma comparação direta com FSMs, mas sim demonstrar a viabilidade de uma estrutura baseada em árvores de comportamento em sistemas embarcados, com foco na simplicidade estrutural, modularidade e uso mínimo de recursos, compatível com ambientes restritos. Essa abordagem pode representar uma alternativa viável para uma aplicação embarcada, respeitando o comportamento e a lógica geral da árvore de comportamento, mas tentando observar outras perspectivas de abstração para conseguirem trabalhar de forma eficiente em um sistema embarcado. O artigo "Comparison between Behavior Trees and Finite State Machines"(2024) aborda uma comparação de FSMs com árvores de comportamento de forma bem detalhada.

1.2 Objetivos

Avaliar a viabilidade e a implementação de uma estrutura básica de uma árvore de comportamento na linguagem de programação C, mantendo seu comportamento determinado.

1.2.1 Objetivos específicos

- Processamento resumido de uma árvore de comportamento;
- Explorar uma abordagem independente de FSM, destacando a aplicabilidade modular da árvore de comportamento em um sistema embarcado;
- Garantir o mínimo de recurso possível de qualquer *RTOS*.

2

Fundamentação Teórica

Neste capítulo, são abordados os conceitos fundamentais para a compreensão do funcionamento de uma árvore de comportamento, visando sistemas embarcados. Conceitos esses que servem como base para a estrutura em sistemas embarcados.

2.1 Grafos

Grafos são estruturas usadas de forma representativa em modelos matemáticos para relacionar elementos. Um grafo é definido por um conjunto V não vazio e finito de vértices (ou nós) e um conjunto E de arestas, que estabelece relações binárias entre esses vértices (LEVADA, 2022).

Formalmente, um grafo G é definido como:

$$G = (V, E)$$

onde:

- V é um conjunto de vértices, tal que $V \neq \emptyset$;
- E é um conjunto de arestas, onde $E \subseteq V \times V$, ou seja, representa relações binárias entre os elementos de V.

Quanto às suas classificações, podem ser direcionados ou não direcionados. Os grafos são direcionados quando as arestas possuem uma direção específica, indicando uma transição entre os vértices. Já os não direcionados, sua aresta é bidirecional, indicando assim que os vértices podem ir de um para o outro.

A ponderação é uma propriedade opcional que estende a estrutura básica de um grafo, associando valores numéricos (pesos) a arestas ou vértices, sem alterar sua classificação, adicionando apenas informações. Formalmente, para um grafo $G = (V, E)$, define-se:

- **Ponderação em arestas**: Uma função $w\colon E \to \mathbb{R}$ que mapeia cada aresta a um peso (custo, distância, requisito, etc).
- **Ponderação em vértices**: Analogamente, uma função $w_V\colon V \to \mathbb{R}$ associa pesos aos vértices.

Um aspecto importante para grafo é a existência ou não de ciclos. Um ciclo ocorre quando partindo de um nó arbitrário, é possível retornar para ele devido a ligação das arestas ao longo do grafo, ou seja, um ciclo num grafo é um caminho fechado (FEOFILOFF, 2020), onde em termos de algoritmos, sua execução podem ser infinitas. Por outro lado, um grafo acíclico sem ciclos, possui sempre um caminho fechado entre seus vértices, chamados também de nós folhas (NEEDHAM; HODLER, 2019), ou seja, existe sempre um início e um fim, sendo dito como um grafo especial (LEVADA, 2022).

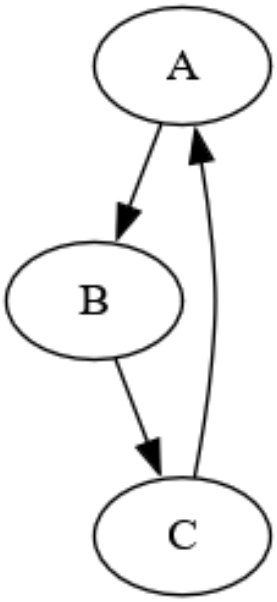


Figura 1: Grafo cíclico.
Fonte: Elaborado pelo autor, (2025)

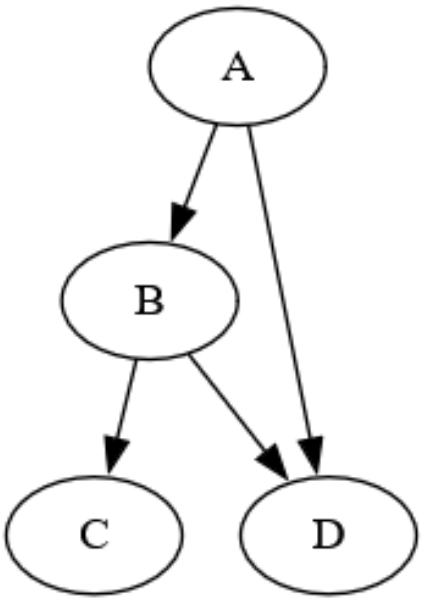


Figura 2: Grafo acíclico.
Fonte: Elaborado pelo autor, (2025)

Um grafo acíclico é também conhecido como o Grafo Acíclico Dirigido (*Directed Acyclic Graph*, *DAG*). Suas arestas são sempre dirigidas a um outro vértice, tendo sempre um *dead end* (NEEDHAM; HODLER, 2019), permitindo uma ordem topológica e uma estrutura hierárquica entre os vértices, possuindo sempre uma sequência de dependências. Essa característica é essencial para modelar sistemas com fluxos unidirecionais e sem retroalimentações, como em fluxo de trabalho ou redes de dependência.

A complexidade de uma DAG é de $O(n+m)$, onde "n" representa o número de nós (vértices) do grafo e "m" o número de arestas, considerando que suas arestas também são processadas de forma isolada. Essa complexidade se dá porque sua estrutura é topológica, o que pode permitir que cada nó e aresta sejam visitados ao menos uma vez, resultando em uma complexidade linear,

considerando o fato de vértices e arestas sendo processadas de forma independente um do outro. Segundo Needham e Hodler (NEEDHAM; HODLER, 2019), essa complexidade é considerada eficiente para muitas aplicações práticas, especialmente quando o DAG possui uma estrutura topológica e hierárquica bem definida. A ordenação topológica, por exemplo, garante que as dependências entre os nós sejam resolvidas de forma sequencial e sem retroalimentação, o que simplifica a execução de algoritmos e reduz a necessidade de processamento adicional. Essa característica torna os DAGs ideais para modelar sistemas com fluxos unidirecionais, como redes de dependência, fluxos de trabalho e agendamento de tarefas.

2.2 Árvores

Segundo Needham e Hodle (2019), na teoria dos grafos, um grafo acíclico não direcionado é chamado de árvore. Na computação, no entanto, essas árvores também podem ser representadas de forma direcionada. Uma definição mais abrangente para uma árvore, é aquela em que quaisquer dois nós estão conectados por apenas uma aresta, ou seja, só há um caminho entre eles. A formação de uma estrutura hierárquica é bem mais evidente em árvores. Elas podem ser classificadas em diversos tipos, de acordo com suas características e funcionalidades.

Uma característica fundamental de uma árvore é sua estrutura hierárquica, onde cada nó (exceto o raiz) tem somente um nó pai, formando uma estrutura sem ciclos. Um DAG também pode ter uma estrutura hierárquica e ser ordenado topologicamente, mas nem todo DAG é uma árvore. Isso ocorre porque um DAG permite que nós tenham múltiplos pais ou que existam múltiplos caminhos entre nós, desde que não haja ciclos. Por outro lado, toda árvore é um tipo específico de DAG, já que uma árvore é um grafo acíclico e dirigido.

As árvores são amplamente utilizadas em computação devido à sua eficiência em representar relações hierárquicas e dependências. Elas podem ser classificadas em diversos tipos, dependendo de suas características e funcionalidades. Alguns exemplos incluem:

1. **Árvores binárias**: Cada nó possui no máximo dois nós filhos, sendo bastante usadas em algoritmos de busca e ordenação;

2. **Árvores de decisão**: Utilizadas em algoritmos de aprendizado de máquina e tomada de decisão, onde cada nó representa uma decisão ou um critério de divisão, e os nós folhas representam os resultados finais;

3. **Árvores de comportamento**: Define um comportamento de uma entidade, onde suas folhas são ações ou checagens de ações e os nós intermediários representam políticas e controles do fluxo.

As árvores de comportamento são muito utilizada nas industrias dos jogos eletrônicos, os NPCs (*Non-Player Characters*) desempenham um papel fundamental na imersão e na dinâmica

dos jogos. Para que esses personagens interajam de forma coerente com o ambiente e com o jogador, é necessário que possuam um certo grau de inteligência. Essa inteligência se manifesta em ações como adaptar sua dificuldade ao jogador, caminhar pelo cenário, abrir portas ou simplesmente permanecer em determinada posição. Embora essas ações possam parecer simples individualmente, sua implementação em grande escala, com dezenas, centenas ou até milhares de NPCs, adiciona uma complexidade significativa ao desenvolvimento do jogo.

2.2.1 Uma necessidade comportamental

Segundo Thompson (2023), em 1998, um jogo chamou bastante atenção na industria, *Half-Life*, onde seus NPCs tinham uma certa inteligência, onde os mesmos podiam mudar seus comportamentos adaptáveis ao ambiente, ou seja, uma entidade poderia ter vários comportamentos diferentes dependendo do ambiente, item próximo, localização do jogador, etc. Outros jogos antes tentaram o mesmo, porém apenas dando espaço para novas tecnologias e estabelecimento de padrões para os jogos, como era feito nos jogos da Nintendo 64.

Tradicionalmente, métodos como máquinas de estados finitos e redes de Petri eram utilizados para modelar o comportamento dos NPCs. No entanto, essas técnicas apresentavam desafios significativos, pois dependendo do caso, uma única transição de ação pode levar a diversas alterações de estados ou eventos. Um exemplo clássico é um NPC que precisa patrulhar uma área em um espaço aberto, ele pode simplesmente correr em linha reta em direção ao jogador, mas, ao transitar para um ambiente mais complexo, como um laboratório com vários obstáculos, a abordagem linear se torna inadequada.

Para lidar com isso, os NPCs precisavam conhecer os pontos acessíveis do mapa (*waypoints*) e adaptar sua navegação conforme o cenário. O problema surge quando, em uma máquina de estados, é necessário gerenciar a transição entre eles de forma eficiente, caso contrário, podem ocorrer falhas lógicas, como um NPC atacando indefinidamente uma porta indestrutível apenas porque o jogador está atrás dela. Isso acontece devido à concorrência de eventos dentro da estrutura da máquina de estados, tornando o sistema propenso a bugs e exigindo um grande volume de código específico para cada interação com o ambiente, o que reduz a modularidade da solução.

Pensando nisso, em 2004, o jogo *HALO 2*, trouxe uma nova ferramenta para auxiliar nessas montagens de comportamentos de NPCs, onde muitos podem ter comportamentos semelhantes, como por exemplo, caminhar para frente (MICROSOFT, 2023). A ferramenta pensada em questão, chamada de *Behavior Tree* (Árvore de Comportamento), se mostrou superior a outras ferramentas usadas na época, como máquinas de estados ou redes de petri. Pensada inicialmente nos cenários de jogos, árvores de comportamentos foram criadas para entidades, sistemas ou aplicações, onde a modularidade, hierarquia, organização, simplicidade, reatividade e sintaxe simples fossem a base de sua aplicação, não variando de acordo com o cenário, e que apenas a junção deles alterem o comportamento geral, como um personagem patrulhar em um cenário

sem e com obstáculos os comportamentos sejam os mesmos, pois personagem e ambiente são entidades diferentes com comportamentos únicos, onde a entidade personagem pode se adequar ao ambiente. A única coisa que deve ser reportado a patrulha, é apenas os *waypoints* do cenário, assim duas entidades conseguem trocar informações enquanto são executadas em paralelo, corrigindo *bugs* de várias combinações diferentes das entidades, pois todas entidades agora são modulares.

Esses *bugs* são provenientes da estrutura de uma FSM, pois a adição de um simples vértice, adiciona uma complexidade a mais na aplicação, onde esse vértice consegue transitar para qualquer outro vértice da FSM, adicionando $|V| - 1$ arestas, onde $|V|$ é o número de vértices da FSM. Com isso, pode surgir casos específicos em certas transições que devem ser tratadas, mas que tratar tudo isso pode ser exaustivo e complexo, como pode ser visto na figura 3, onde temos uma FSM apenas com o tratamento de que o evento seguinte não é o mesmo que o anterior, com apenas três vértices, e na figura 4, com a adição de apenas um nó *D*, é adicionado uma nova série de arestas ao grafo.

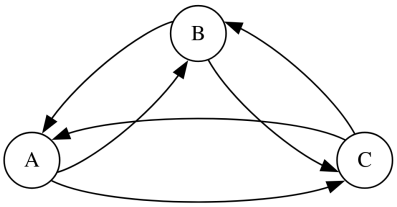


Figura 3: FSM antes de modificação.
Fonte: Elaborado pelo autor, (2025)

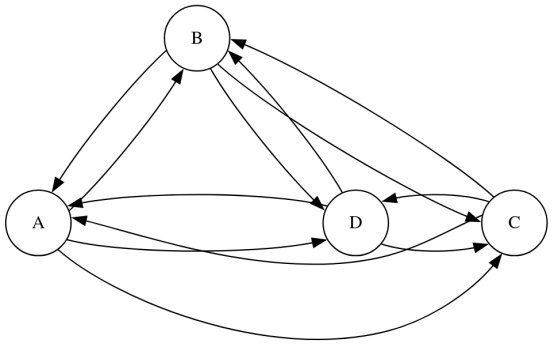


Figura 4: FSM com a implementação do nó *D* adicional.
Fonte: Elaborado pelo autor, (2025)

Devido às suas vantagens estruturais, não demoraram a ser adotadas em outras áreas além dos jogos eletrônicos. Na robótica, por exemplo, onde a modularidade e a previsibilidade dos sistemas são essenciais, essa abordagem se mostrou altamente eficaz. A robótica, assim como os jogos, exige sistemas adaptáveis e reativos, capazes de lidar com múltiplas entradas simultâneas sem comprometer a estabilidade da aplicação. Dessa forma, as árvores de comportamento se consolidaram como uma solução robusta tanto para inteligência artificial em jogos quanto para sistemas autônomos na indústria.

2.3 Árvore de comportamento

Proposta inicialmente por Brooks (1986), trata-se de uma estrutura abordada em forma de lista de comportamentos voltada ao controle hierárquico de sistemas autônomos, onde logo foi repensada para funcionar como uma árvore, devido a sua estrutura semelhante a uma árvore de decisão. A árvore de comportamento é um modelo matemático hierárquico de controle amplamente utilizado em inteligência artificial para planejamento e execução de sequências de ações, especialmente em jogos e robótica (COLLEDANCHISE; ÖGREN, 2018). Também aplicada à teoria da decisão em sistemas automatizados.

Diferente das outras estruturas, principalmente das máquinas de estados, seu comportamento é previsível e sequencial, podendo ser bem planejada para realizar procedimentos em pequenos blocos, além de ser bastante intuitiva em relação a outras técnicas, podendo mudar todo o contexto em pequenos passos, evitando possíveis *bugs* de um evento descartado durante o processo, pois nas máquinas de estados, uma troca rápida de eventos que poderia causar erros e bugs graves na aplicação, pois a transição de eventos e estados é brusca, e se não for bem planejada, itens que não são consideradas em certos eventos são alterados.

As principais filosofias são:

- **Hierarquia:** Há uma ordem hierárquica que define a ordem do que deve ser executado;
- **Reuso:** Promovem modularidade ao dividir o comportamento em ações simples e reutilizáveis;
- **Organização:** Estruturas bem definidas e intuitivas fáceis de desenhar e manusear;
- **Reatividade:** Capacidade de se adaptar dinamicamente a mudanças no ambiente, garantindo respostas rápidas e eficientes.

As árvores de comportamento podem ser divididas em ***nós***, ***estruturação***, e ***blackboard***.

2.3.1 Nós

Os nós são os blocos fundamentais de construção de uma árvore de comportamento, representando os elementos estruturais que definem seu funcionamento, atuam como unidades modulares que compõem a árvore, determinando o fluxo da execução e influenciando diretamente o comportamento do sistema.

Cada nó possui uma função específica dentro da árvore e estabelece conexões que definem como as decisões são tomadas e as ações são executadas. A organização dos nós permite a criação de estruturas flexíveis e escaláveis, garantindo que comportamentos complexos sejam representados de maneira clara e eficiente.

A estruturação dos nós é essencial para a previsibilidade e controle da árvore, permitindo um design hierárquico e modular. Essa abordagem possibilita ajustes, reutilização de componentes e facilita a manutenção, tornando as árvores de comportamento uma ferramenta poderosa para modelagem de comportamentos dinâmicos.

Os nós podem ser classificados nas seguintes categorias: ***raiz***, ***execução***, ***controle*** e de ***decorator***.

* **Raíz**

  O nó raiz só pode ter um filho pelo simples fato de que deve ter apenas uma ramificação para iniciar e finalizar toda a árvore.

* **Nós de execução**

  Os nós de execução, também conhecidos como nós folhas ou nós de interação, são os responsáveis por executar blocos de funções. Estes blocos são fragmentos de funcionalidades, carregando um fragmento da funcionalidade da aplicação, determinando o funcionamento como um todo.

  Os blocos de execução não devem ser muito extensos, pois sua principal função é dividir a lógica em pequenos blocos, facilitando o reaproveitamento dentro da aplicação e até mesmo a portabilidade para outros sistemas. Esse reaproveitamento é um dos principais benefícios da árvore de comportamento, permitindo maior flexibilidade e simplicidade na manipulação do comportamento da aplicação.

  Os nós de execução são divididos principalmente em dois tipos:

  - **Condição:** Verifica se uma ou mais condições foram satisfeitas. Esse tipo de nó apenas deve retornar sucesso ou falha, visto a sua função é de validação;
  - **Ação:** Executa um trecho de código da aplicação, carregando uma ou mais funcionalidades do sistema. Para manter o sistema modular e organizado, é recomendado que cada nó de ação implemente apenas um tipo de funcionalidade. Esse nó pode retornar sucesso, falha ou *running*, caso a execução da ação exija mais tempo.

* **Nós de controle**

  Os nós de controle são responsáveis por gerenciar a execução dos nós filhos determinando as condições e a lógica de execução da árvore atuando como intermediários que organizam o fluxo da execução, decidindo quais nós serão chamados.

  Esse tipo de nó é essencial para estruturar o comportamento da árvore de forma hierárquica e organizada, avaliando o estado dos seus filhos e determinando como a árvore deve reagir às diferentes situações, garantindo que as ações ocorram de maneira ordenada e eficiente.

São a base para a implementação de lógica condicional e sequencial, possibilitando que um sistema tome decisões dinâmicas com base no contexto e nas condições estabelecidas.

Os nós de controle permitem a criação de comportamentos mais complexos a partir da combinação de ações menores, tornando a árvore mais modular e adaptável.

Os nós de controle são divididos em três principais tipos:

– **Sequência ou *Sequence*:** Executa seus filhos em ordem até que um deles falham, retornando falha, ou todos os seus filhos tiverem sucesso, retornando sucesso. Seu símbolo é "$\longrightarrow$";

Pseudocódigo:

Algoritmo 1: Nó Sequence (Comportamento Sequencial)

Entrada: filhos $(filho_1, filho_2, ..., filho_n)$
Saída: Status (Sucesso, Falha ou Executando)

```
1 Para filho in filhos faça
2     status ← tick(filho);
3     Se status = Failure então
4         Retorne Failure;
5     Se status = Running então
6         Retorne Running;
7 Retorne Success;
```

– **Seletor ou *Fallback*:** Executa seus filhos até que algum seja sucesso e retornando sucesso. Caso todos falhem, é retornada a falha. Seu símbolo é "?";

Pseudocódigo:

Algoritmo 2: Nó Fallback (Comportamento de *Fallback*)

Entrada: filhos $(filho_1, filho_2, ..., filho_n)$
Saída: Status (Sucesso, Falha ou Executando)

```
1 Para filho in filhos faça
2     status ← tick(filho);
3     Se status = Sucesso então
4         Retorne Sucesso;
5     Se status = Executando então
6         Retorne Executando;
7 Retorne Fail;
```

– **Paralelo ou *Parallel*:** Executa seus filhos simultâneamente e retorna sucesso se, e somente se, um número definido de filhos forem sucesso, caso contrário, falha. Seu símbolo é "$\rightrightarrows$";

Pseudocódigo:

Algoritmo 3: Nó Paralelo (Comportamento Paralelo)

Entrada: filhos $(filho_1, filho_2, ..., filho_n)$, política de sucesso K

Saída: Status (Sucesso, Falha ou Executando)

```
1  sucessos ← 0;
2  falhas ← 0;
3  Para filho in filhos faça
4      status ← tick(filho);
5      Se status = Sucesso então
6          sucessos ← sucessos + 1;
7          Se sucessos ≥ K então
8              Retorne Sucesso;
9      Se Não Se status = Falha então
10         falhas ← falhas + 1;
11         Se falhas > (n - K) então
12             Retorne Falha;
```

* Nós decorators

Os nós *decorators* são responsáveis por modificar o comportamento de um nó ou ramificação, atuando como intermediários, alterando a lógica de execução ou o retorno do nó, permitindo um maior controle e flexibilidade dentro de uma árvore de comportamento, sendo úteis para criação de variações no comportamento sem a necessidade de modificar os nós de ação ou condição, tornando a árvore reutilizável e modular, sem a necessidade de modificar a estrutura principal da aplicação, o que a deixa flexível e dinâmica.

Existem vários tipos de nós *decorators*, mas os principais são:

- **Inversor:** Inverte o resultado do nó filho ou da ramificação. Se a ramificação retorna sucesso, ela altera para falha e vice-versa. Caso retorne running, o inversor mantém esse estado;
- **Sucesso/Falha Forçado:** Força o retorno de sucesso ou falha, independentemente do resultado do nó filho ou da ramificação;
- **Repetidor:** Executa o nó filho repetidamente, conforme um número pré-definido de vezes ou até que uma condição seja atendida;

2.3.2 Tick

Um *tick* é um sinal de requisição de execução de um nó, propagado pela raiz de uma árvore, ou seja, do seu ponto inicial até um outro nó na árvore. Um nó de fato só é executado quando a árvore envia um *tick* para ele, chamando-o para executar, sendo assim este *tick* carrega informações pertinentes para o nó alvo, carregando seu status de execução através da árvore a quem interessar o estado do nó, como:

- ***Success*:** Quando um nó tem sua execução finalizada de forma satisfatória ou que sua condição seja satisfeita;
- ***Fail*:** Quando um nó tem sua execução finaliza de forma incompleta ou que sua condição não seja satisfeita;
- ***Running*:** Quando um nó não pode finalizar sua execução ainda, pois depende de alguns fatores como tempo para tarefas duradouras ou execução não finalizada de seus filhos;
- ***Error*:** Quando há alguma falha na árvore que não é prevista, como má formação da árvore, ou um defeito na estruturação da mesma, sendo necessário uma manutenção ou refazer a árvore;

Algumas implementações abordam *ticks* como uma execução de um único bloco apenas, independente de qual seja. Outros algoritimos já integram as execuções intermediárias como parte de um único *tick*.

2.3.3 Estruturação

A estruturação desempenha um papel fundamental no funcionamento da árvore de comportamento, pois é por meio dela que a árvore é organizada, analisada e interpretada para gerar os comportamentos desejados. Essa estruturação define a cadeia de decisões que serão tomadas e quais ações serão executadas, executando cada bloco individualmente e ordenadamente, dividindo a aplicação em ramificações de acordo com o comportamento desejado.

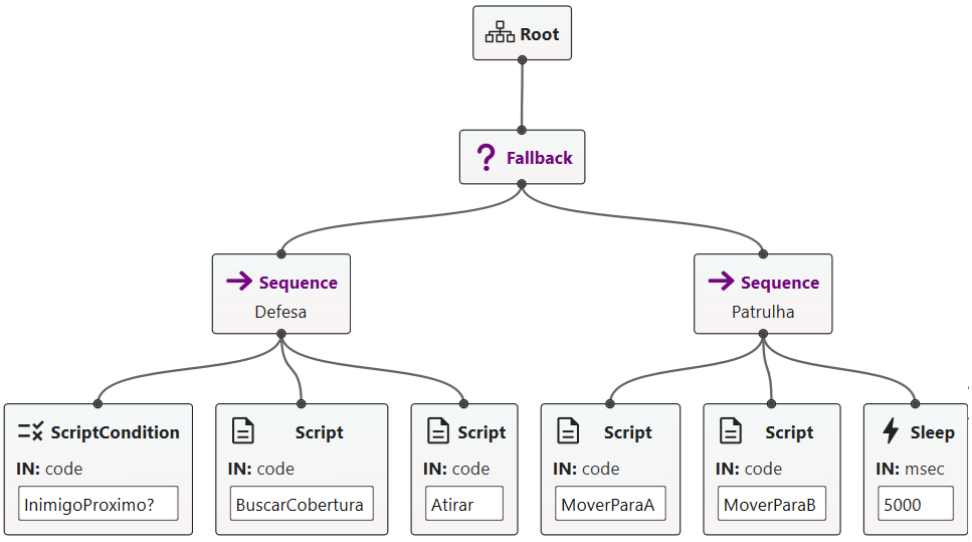


Figura 5: Árvore de comportamento simples.
Fonte: Elaborado pelo autor, (2025)

Na Figura 5 vemos uma árvore de comportamento hierárquica e lógica, aplicada a um personagem de jogo com três rotinas: **defesa** e **patrulha**. A execução segue a estrutura da árvore, começando pelo nó raiz (*Fallback*) e percorrendo os nós da **esquerda para a direita**, em profundidade.

O comportamento da árvore da figura 5 é executado da seguinte forma:

1. **Nó *Root*:**
   - Verifica o status da árvore.
2. **Nó *Fallback*:**
   - Verifica seus filhos em ordem até que um retorne **sucesso**;
   - Se um filho falhar, passa para o próximo.
3. **Ramificação *Defesa* (*Sequence*)**:
   - **Condição:** Verifica se há um inimigo próximo (*InimigoProximo?*):
     - **Se verdadeiro:**
       - Executa *BuscarCobertura* (movimento para proteção);
       - Em seguida, *Atirar* (ação de ataque).
     - **Se falso:** Falha e o *Fallback* avança para *Patrulha*.
   - Se todos os passos forem bem-sucedidos, a *Sequence* retorna **sucesso**, finalizando a árvore.
4. **Ramificação *Patrulha* (*Sequence*)**:

- Só executa se *Defesa* falhar;
- Move o personagem entre pontos pré-definidos (*MoverParaA*, *MoverParaB*);
- Aguarda 5 segundos (*Sleep*) antes de repetir.

A ordem de prioridade se dá da esquerda para a direita, ou seja, da ramificação *Defesa* para a *Patrulha*, onde se a *Defesa* for bem-sucedida, a árvore encerra imediatamente, sem executar *Patrulha*, caso contrário, a ramificação *Patrulha* deve ser executada, conforme a lógica do *Fallback*. Esta estrutura garante comportamentos adaptáveis, onde ações críticas (como defesa) interrompem tarefas menos urgentes, refletindo decisões inteligentes e eficientes.

2.3.4 Blackboard

A estruturação de um sistema em blocos individuais permite uma execução modular e organizada, tornando cada componente independente e reutilizável. Essa abordagem evita a comunicação direta entre os módulos, o que otimiza a coesão e reduz o acoplamento. Para a comunicação entre os comportamentos, é necessário um mecanismo eficiente que permita a troca de informações entre os nós da árvore sem comprometer sua independência.

A blackboard surge como um método de comunicação dos nós, funcionando como uma interface centralizada para armazenar e compartilhar dados relevantes dentro da árvore de comportamento. Em vez de permitir que os nós se comuniquem diretamente entre si, o que pode gerar dependências, a blackboard organiza os dados de maneira acessível e estruturada. Dessa forma, informações globais podem ser compartilhadas entre diferentes partes da árvore, permitindo a modularidade da aplicação, facilitando acessos a dados em diferentes comportamentos, permitindo expansões futuras sem comprometer sua estrutura original.

2.4 Complexidade da Árvore de comportamento

Semelhante aos DAGs, a execuxão de uma árvore de comportamento possui complexidade $O(n)$, onde "n"representa o número de nós que contém na árvore. Essa complexidade linear é devido sua estrutura hierárquica e topológica, permitindo a execução de cada nó de forma ordenada e sem ciclos permanentes. Embora a árvore de comportamento possa conter ciclos, eles são projetados para serem finalizados após um número definido de tentativas, garantindo que a execução siga regras preestabelecidas e não entre em *loops* infinitos.

Apesar da existência dos nós decorators, que podem realizar comportamentos repetitivos, eles apenas servem para expandir a árvore de forma modular, evitando a criação de ramificações excessivamente complexas ou extensas, permitindo a reutilização de lógica sem comprometer a simplicidade e a eficiência da estrutura geral da árvore, como pode ser vista nas figuras 6 e 7.

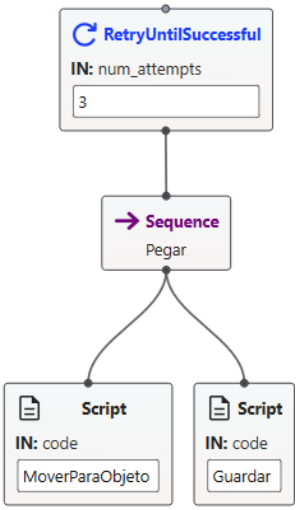


Figura 6: Ramificação repetitiva.
Fonte: Elaborado pelo autor, (2025)

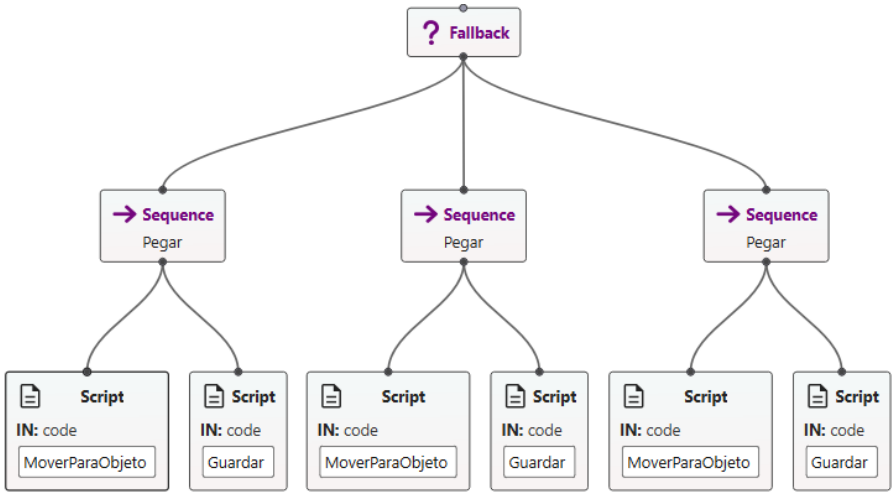


Figura 7: Extensão da ramificação.
Fonte: Elaborado pelo autor, (2025)

2.5 Modelagem de árvores de comportamento

A modelagem visual de estruturas hierárquicas, como árvores de comportamento, é comumente usada em desenvolvimento de projetos em geral, como diagramas, fluxogramas, etc. Utilizar ferramentas apropriadas para a criação de diagramas permite não apenas representar graficamente o comportamento do sistema, mas também facilita o processo de análise, manutenção e expansão do sistema.

Uma ferramenta para modelagem árvores de comportamento é o Groot2 (2024), por possuir mecanismos de modelagem destas estruturas, além de mitigar erros de modelagem no projeto. O Groot2 destaca-se como uma solução eficiente, por se tratar de um *software* prático e gratuito (versão básica), desenvolvido especificamente para construção e simulação de árvores de comportamento. Seu uso é voltado a desenvolvimento na linguagem *C++*, contudo utilizare-

mos apenas os recursos de diagramação e padronização do formato dos arquivos construídos na ferramenta.

Um dos principais diferenciais da ferramenta é a validação automática da estrutura antes da exportação ou salvamento: árvores com conexões inválidas, como nós desconectados ou ligações incorretas, são sinalizadas, impedindo o salvamento do projeto até que os problemas sejam resolvidos. Essa funcionalidade ajuda a prevenir falhas lógicas e topológicas já na fase de modelagem. Um exemplo de árvore projetado nessa ferramenta, está representada na figura 5.

Ao adotar o Groot2 no processo de desenvolvimento, é possível obter não apenas uma representação visual e padronizada das árvores de comportamento, mas também garantir a consistência estrutural do projeto desde as primeiras etapas, onde sua saída será um arquivo XML (*Extensible Markup Language*, linguagem de marcação que guarda dados em formato hierárquico, legível por humanos e máquinas) que contará com a árvore de comportamento.

Após a exploração dos conceitos da teoria das árvores de comportamento e a ferramenta de modelagem, chega-se ao cerne da proposta, uma estrutura embarcada. Este campo impõe desafios consideráveis devido às suas restrições de recursos, exigindo adaptações dessas estruturas para uma melhor eficiência. Por isso, no próximo capítulo, dedica-se a descrever em detalhes as estratégias e técnicas que permitem otimizar a implementação das árvores de comportamento, garantindo sua performance e viabilidade neste cenário.

3

Metodologia

Neste capítulo, será descrito a metodologia no desenvolvimento do projeto, os tecnologias usadas para desenvolvimento, bem como as plataformas utilizadas para o desenvolvimento e testes práticos em diferentes MCUs. A abordagem metodológica contempla a problemática da implementação de uma árvore de comportamento em C, propondo novas soluções que consideram o desempenho e o uso de memória, aspectos essenciais para sistemas embarcados.

3.1 Recurso embarcado

Em sistemas embarcados, os recursos computacionais dispostos são normalmente limitados, podendo ser restritos da classe 0 (SHELBY et al., 2014), a exemplo o microcontrolador ATMega328PB, com 2 KB e EEPROM de 1 KB, ou até hardwares da classe 2, a exemplo microcontroladores da família STM32L476xx, com 128kB de RAM e 1MB de EEPROM. Devido a essas restrições, o desenvolvimento de software para tais plataformas deve priorizar:

- **Otimização:** O código deve ser minimalista, evitando alocações dinâmicas e estruturas complexas;
- **Modularidade:** A estruturação do código deve permitir a reutilização de componentes em diferentes contextos em rotinas e em *drivers*, possibilitando o acoplamento mínimo de rotinas;
- **Padronização:** A padronização do código deve ser facilitada para a manutenção e o entendimento por parte da equipe de desenvolvimento.

Nesse contexto, a metodologia para desenvolvimento de *software* em sistemas embarcados deve priorizar eficiência e organização. A abordagem com o uso de árvores de comportamento em sistemas embarcados se torna relevante para a organização e fluxo linear de execução, oferecendo uma hierarquia de componentes e tarefas e conseguindo abordar lógicas complexas de forma eficiente e modular, mesmo que esteja disponível poucos recursos.

3.2 Desafios de uma árvore de comportamento embarcada

As árvores de comportamento podem apresentar desafios significativos ao serem aplicadas diretamente em sistemas embarcados. Caracterizados por restrições de memória, processamento e requisitos de tempo real, certas características podem levar a um alto consumo de recursos nas árvores de comportamento pode ser causado por:

- **Flexibilidade:** Um nó de controle pode ter um número variável de filhos, o que pode exigir uma alocação dinâmica de dados ou estruturas pré-dimensionadas;
- **Estrutura:** A estrutura de uma árvore pode custar muito processamento, devido a revisitação de nós já visitados, ou que não precisam ser executados.
- **Recursividade:** A execução de um modelo de árvore de comportamento convencional é recursiva, ou seja, um nó chama um outro nó até um nó folha ser executado, e quanto maior a profundidade da árvore, maior o consumo de recursos alocados para a sua execução.

A maioria dos sistemas embarcados utiliza a linguagem C como base. Embora estruturas com itens dinâmicos são viáveis, mas podem não ser bem otimizadas, visto que nós de controles poderiam ter um número de filhos não padronizado e elevando o consumo de memória. Portanto, as estruturas das árvores são definidas e criadas em tempo de compilação e, por isso, devem ter tamanho fixo. Além disso, a execuxão de um nó, em um contexto embarcado, deve ser feita de maneira direta, evitando o uso de recursão, e se um nó ainda não for finalizado, deve-se utilizar o *status running* de uma árvore de comportamento, assim evitando uma recursão.

A utilização de ponteiros para uma alocação externa, pode ser custoso e não eficiente na aplicação devido às formas de verificar o tamanho dos vetores, verificar o tipo de nó, a memória onde está localizada esses dados, etc. Além disso, nesse modelo, quando um nó folha é executado, ele retorna ao nó pai carregando seu status de execução. O nó pai, por sua vez, decide a próxima ação com base em sua regra específica. Por exemplo, ao executar um nó de controle, este invoca um nó filho e, após a execução do filho, o nó de controle é executado novamente para processar o resultado.

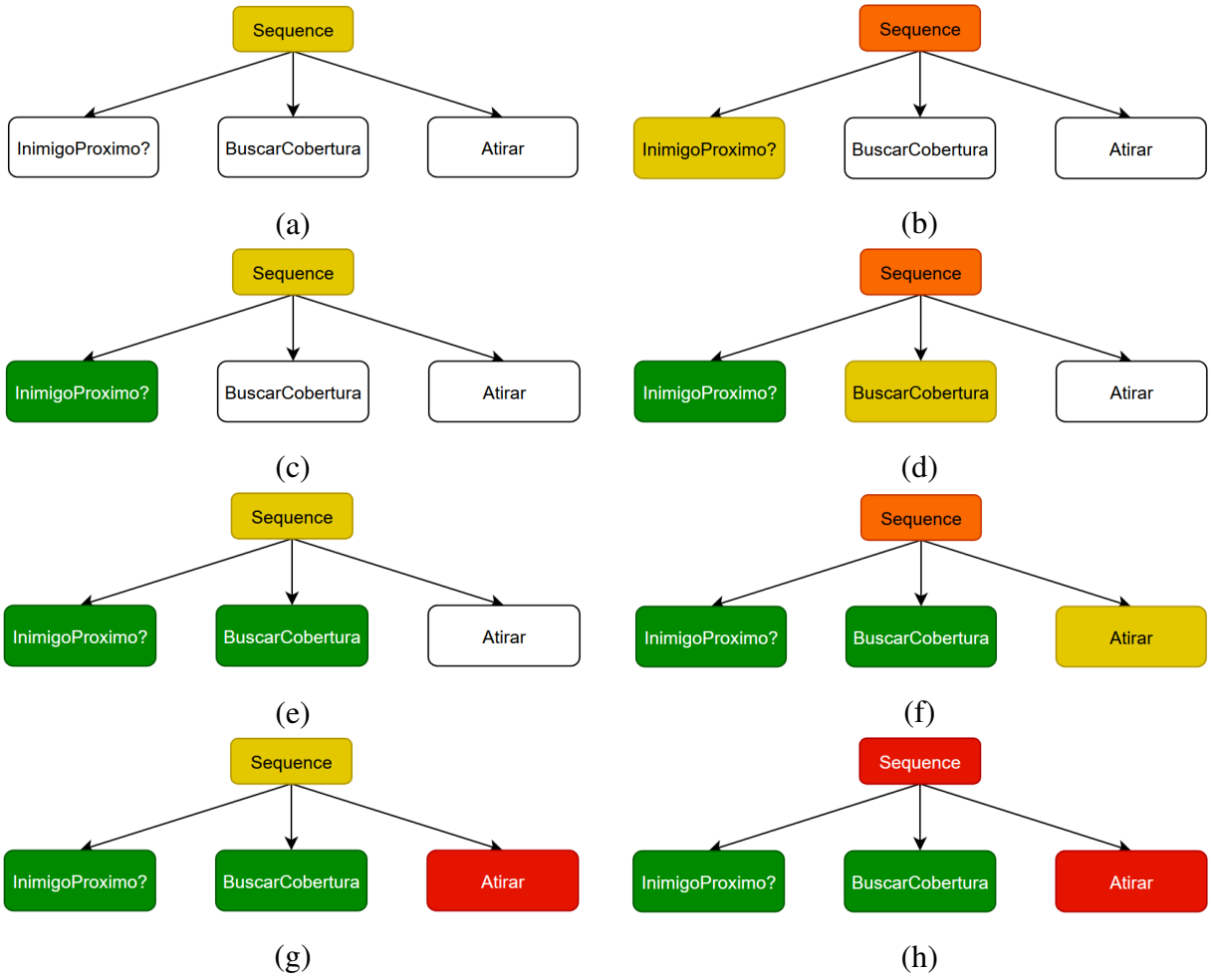


Figura 8: Execução convencional de uma árvore de comportamento

Legenda: Em execução Em espera Sucesso Erro

Fonte: Elaborado pelo autor, (2025)

A execução ilustrada na figura 8 mostra como é uma execução de uma ramificação de uma árvore de comportamento, onde na figura *a* temos o início da execução sendo o primeiro *tick* que já inicia a execução do nó *InimigoProximo?* no mesmo *tick*, sendo finalizado na figura *c* com sucesso. No segundo *tick*, a lógica do nó *Sequence* é reavaliada, que determina que deve continuar a executar seu próximo filho, que é o nó *BuscarCobertura*, na figura *d* e finaliza com sucesso na figura *e*. No terceiro *tick*, seria executado a ação *Atirar*, como retratado na figura *f*, *g* e *h*.

A execução retratada mostra que os nós intermediários são reexecutados a cada avaliação de seus filhos. Em sistemas embarcados, essa reavaliação constante e a travessia frequente de nós para verificar a satisfação da lógica pode ser custosa em termos de processamento e uso de memória para a arquitetura da árvore, necessitando de uma abordagem um pouco mais direta para transitar entre os nós e uma outra abordagem de verificação de lógica aplicada ao nó.

3.3 Proposta prática: Otimização utilizando Pilha e Modelagem K-ária

Uma otimização do árvores de comportamento foi proposta por Alfredo Lima (2022). Sua contribuição primária reside na modelagem da árvore de comportamento, utilizando uma estrutura de árvore binária filha, conforme a representação de Knuth (*left-child*, *right-sibling*). Essa transformação k-ária permite que qualquer árvore com nós de múltiplos filhos seja representada como uma árvore binária, onde cada nó aponta para seu primeiro filho e para seu próximo irmão, como ilustrado na Figura 9.

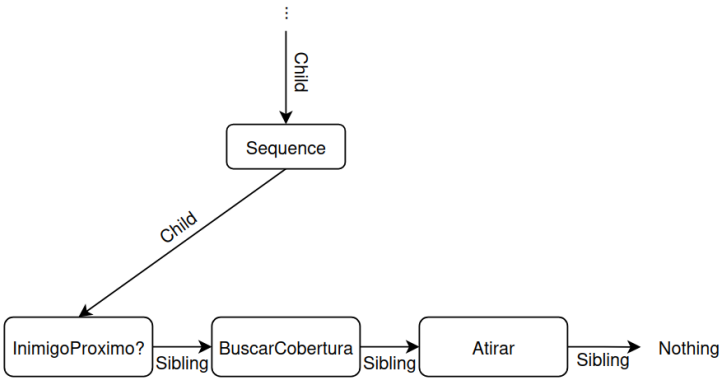


Figura 9: Estrutura com a abordagem de referências dos irmãos.

Nessa abordagem, em vez de depender de recursividade para a travessia da árvore, uma pilha externa é empregada para salvar o contexto de execução. Há apenas referências como nó irmão e nó filho, e os índices desses dois tipos de nós são empilhados. À medida que os nós são processados, verifica-se se ainda há irmãos. O nó processado é desempilhado e o nó no topo da pilha (que agora é um nó de controle) é verificado. Caso a lógica do nó de controle não seja satisfeita, o irmão do nó desempilhado é colocado para ser processado.

Essa simplificação, baseada na transformação de Knuth k-ária e no uso da pilha, permite que a árvore mantenha sua flexibilidade e modularidade, ajustando a estrutura de cada nó individual para direcionar ao nó irmão, deixando a cargo do nó pai decidir se o próximo nó será chamado ou não. Dessa forma, mantém-se a flexibilidade de uma maneira mais simples e constante (no contexto de memória) de ser usada na linguagem C. O uso da pilha fornece uma alternativa eficiente à recursividade, sendo necessário apenas analisar se há itens na pilha, executar, colocar ou remover um nó dela, finalizando sua execução substancialmente ou parcialmente (no caso de nós que retornem *running* ou nós de controle que estejam iniciando sua primeira execução).

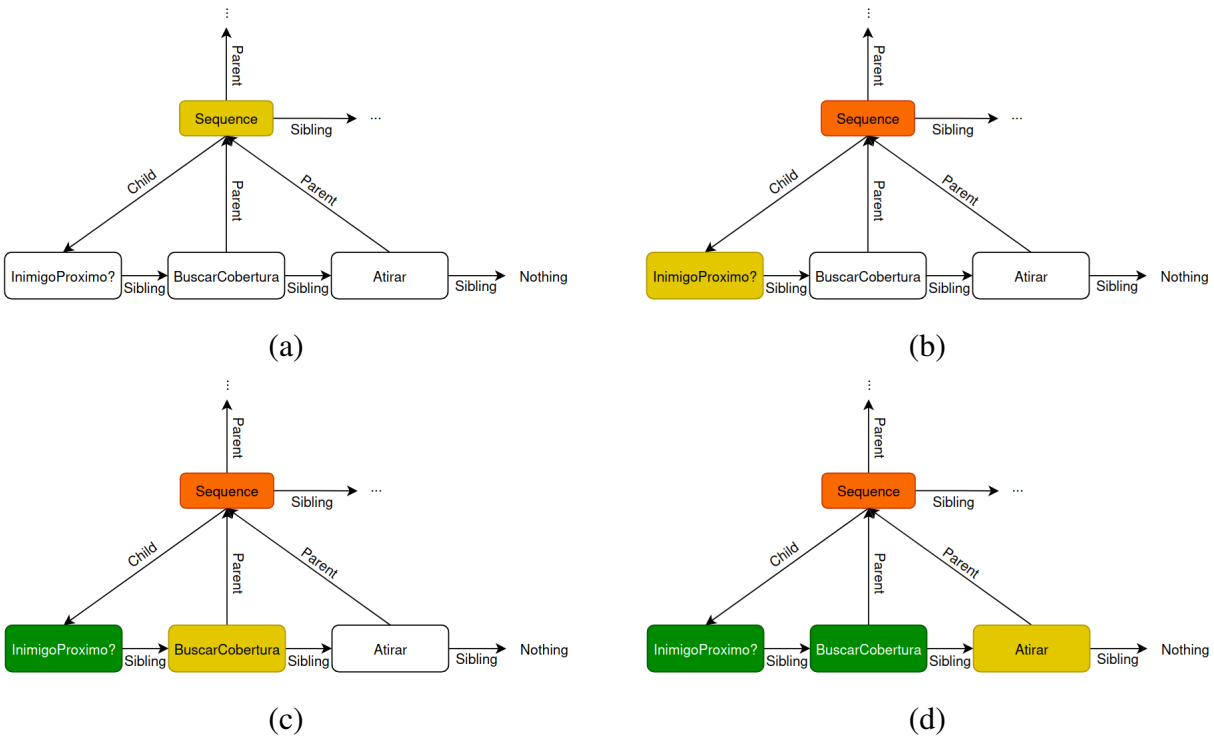
3.4 Proposta prática: Grafo simplificado da estrutura sem pilha

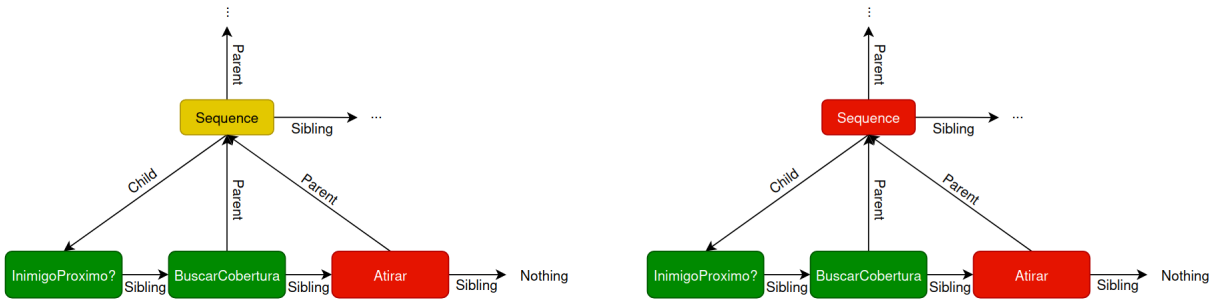
Uma nova abordagem que otimiza a estrutura da árvore de comportamento ao transformá-la em um grafo dirigido. Essa simplificação envolve a adição de novas referências entre os nós, eliminando a necessidade de uma pilha externa para o controle da execução.

Nessa estrutura, cada nó, com exceção do nó raiz, armazena até três referências. Se for um nó de controle ou *decorator*, ele terá referências para seu nó pai, seu primeiro filho e o nó irmão adjacente no mesmo nível à sua direita. Para um nó de execução, apenas duas referências são necessárias: a do nó pai e a do nó irmão.

Essa otimização reduz significativamente as transições de nós durante a execução. Em vez de percorrer toda a estrutura, a regra do nó pai pode ser lida e aplicada sem que haja a necessidade de uma troca explícita para o nó pai, bastando identificar o tipo de nó atual para aplicar sua lógica. O processo de execução cessa quando não há mais nós irmãos a serem processados ou quando a regra do nó pai não é mais válida para o estado atual do nó.

Mesmo com essa simplificação, a identificação dos nós pais permanece fundamental para o fluxo de controle, ainda que não sejam visitados diretamente. Essa abordagem permite que a execução ocorra com menos transições entre os nós, mantendo a mesma quantidade de ticks em comparação com uma árvore de comportamento convencional. A figura 10 ilustra essa execução, que corresponde à apresentada na figura 8.





(e) (f)

Figura 10: Execução simplificada da estrutura.

Legenda: Em execução Em espera Sucesso Erro

Fonte: Elaborado pelo autor, (2025)

O pseudocódigo dessa estrutura reduzida considerando apenas o nó do tipo de interação, *fallback* e *sequence*, pode ser visto abaixo.

Algoritmo 4: Grafo Simplificado da Estrutura

Entrada: Tree, Node, LastState

Saída: Status (Sucesso, Falha ou Executando)

```
1  Se (NodeType = Interator) então
2      Status ← Node→Iteration();
3  Se Não Se (NodeType = Sequence) ∨ (NodeType = Fallback) então
4      Se (LastState = Executando) então
5          Tree→index ← Node→Child;
6          Retorne Executando;
7      Status ← LastState;
8  Se ((ParentIsSequenceAndStatusIsFail(Tree, Node, Status)) ∨
   (ParentIsFallbackAndStatusIsSuccess(Tree, Node, Status)) ∨
   (NotHaveMoreSibling(Tree, Node)))) então
9      LastState ← Status;
10     Tree→index ← Node→Parent;
11 então
12     LastState ← Executando;
13     Tree→index ← Node→Sibling;
14 Retorne Status;
```

A abordagem desse método em comparação com o proposto por Alfredo Lima (2022), retira a necessidade de uma *stack*, mas aumenta o número de arestas e a quantidade de chegagens dos nós intermediários, visto na linha 8 do pseudocódigo 4, o que pode ser uma abordagem

prejudicial se expandir os nós suportados pela aplicação.

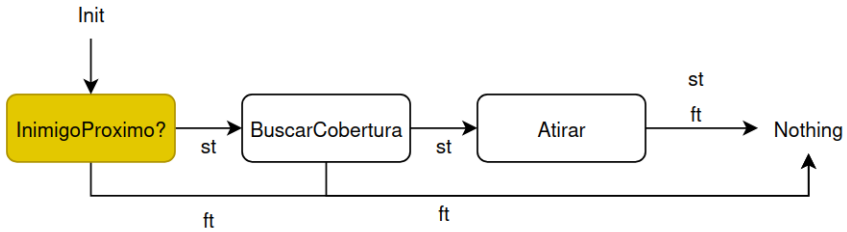
Na perspectiva de memória, há uma vantagem significativa: se utilizarmos a memória *flash* para guardar essas arestas, a memória *RAM* usada pode ser menor do que no método anterior. Pois ao eliminar a *stack*, liberamos a *RAM*, que é um recurso escasso em sistemas embarcados, para outras operações. As referências adicionais seriam armazenadas na *flash*, que é mais abundante e não volátil, tornando a solução mais viável para sistemas embarcados.

3.5 Grafo de comportamento mínimo

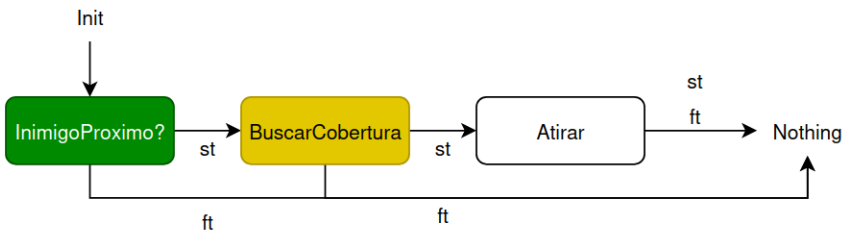
Uma abordagem interessante, em termos de execução, é a exclusão dos nós de controle, como *sequence* ou *fallback*, e a substituição dos tipos de arestas, em comparação com o método citado no tópico 3.4. Isso foi pensado ao analisar que a lógica de ramificação aplicada a um nó, independentemente do seu resultado, sempre o direcionaria para um ponto específico da árvore, verificando que o comportamento da árvore em qualquer nó, tanto sucesso quanto falha, direciona sempre para um caminho descrito pela semântica lógica da árvore.

A aresta do tipo *child* passa a ser convertida em *target*, que representa o alvo de um determinado nó. Esse tipo de aresta é utilizado exclusivamente por nós do tipo *decorator*, já que apenas esses nós possuem um único alvo direto.

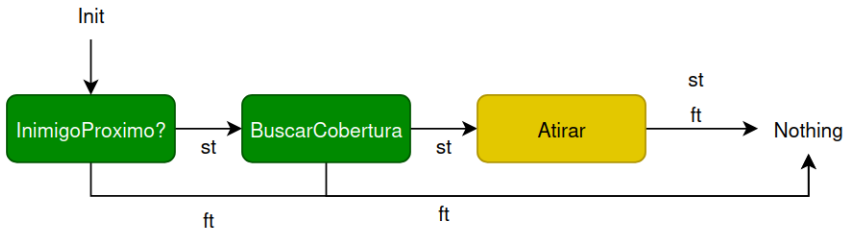
As arestas do tipo *parent* e *sibling* são removidas, pois a exclusão dos nós de controle implica na quebra do fluxo lógico tradicional da árvore. Em seu lugar, são introduzidas as arestas *alvo de sucesso* (*st*), que direciona para um nó caso a execução do nó atual seja bem sucedida, e *alvo de falha* (*ft*), que direciona para um nó caso a execução do nó atual falhe. Essas novas arestas abstraem completamente a semântica das ramificações, permitindo que o fluxo de execução seja direcionado de forma mais eficiente para o próximo nó da aplicação, com base no resultado da execução atual.



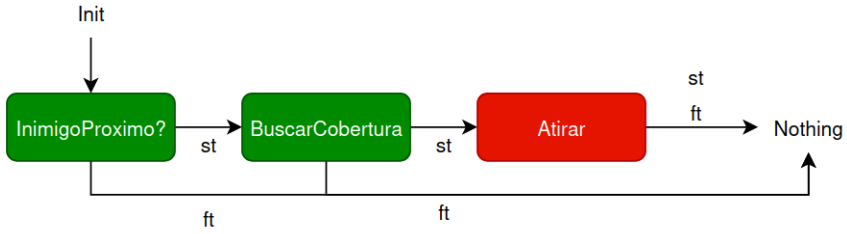
(a)



(b)



(c)



(d)

Figura 11: Execução de uma ramificação com grafo de comportamento mínimo.

Legenda: Em execução Em espera Sucesso Erro

Fonte: Elaborado pelo autor, (2025)

Note que, nesse cenário, a árvore é iniciada diretamente no nó de condição *"InimigoPróximo?"* e finaliza no nó de ação *"Atirar"*, concluindo a execução da ramificação também em três *ticks*, enquanto quantidade de nós executados por *tick* caiu para apenas um nesse cenário. Isso representa uma otimização no processamento da árvore, mantendo o mesmo comportamento funcional, só que de forma simplificada, como ilustrado da figura 11, ilustrado dos itens *a* a *d*.

Apesar da execução e representação estarem distantes de uma árvore de comportamento, na execução é possível pular várias etapas de processamendo sem alterar o resultado final do comportamento da árvore transformada em um grafo de comportamento mínimo, otimizando sua execução, principalmente na troca de ramificações, na qual podem demandar muitos passos, e para trocar uma ramificação para outra é necessário realizar pelo menos dois *ticks* com três execuções de nós intermediários, como pode ser visto na figura 12, nas ilustrações *a* a *e*, enquanto que na estrutura mínima, podemos realizar isso em apenas um *tick* com chamadas diretas, como pode ser visto na figura 13, nas ilustrações *a* e *b*.

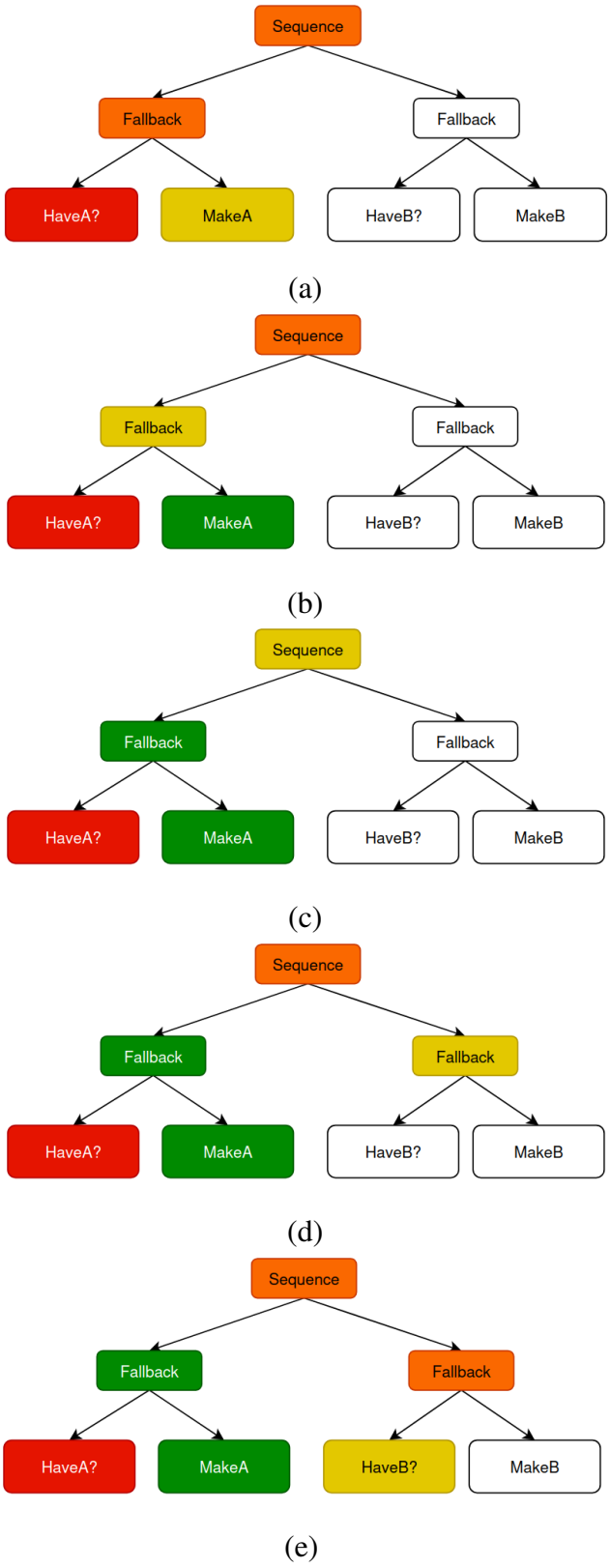
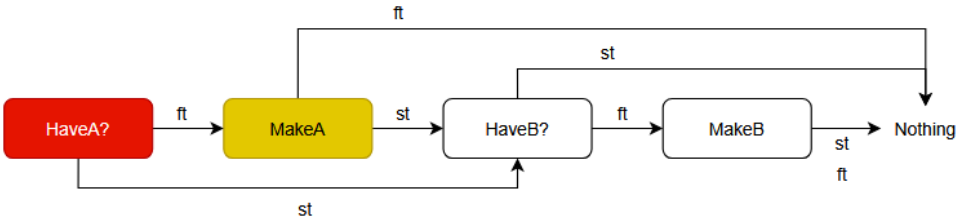


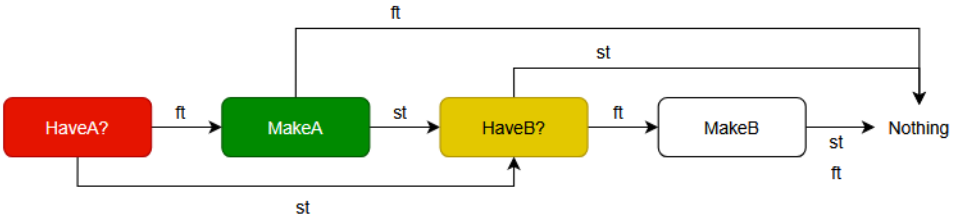
Figura 12: Troca de ramificações de árvore de comportamento.

Legenda: Em execução Em espera Sucesso Erro

Fonte: Elaborado pelo autor, (2025)



(a)



(b)

Figura 13: Troca de ramificações no grafo de comportamento mínimo.

Legenda: Em execução Em espera Sucesso Erro

Fonte: Elaborado pelo autor, (2025)

Agora considerando a representação, para evitar complicações de estruturações, ao utilizar a ferramenta Groot2 para realizar o diagrama da árvore de comportamento e utilizar o seu arquivo de saída para aplicar um pré-processamento de toda a árvore e convertê-la para um grafo de comportamento mínimo, é possível abstrair a sua representação final.

O pseudocódigo dessa estrutura mínima considerando apenas o nó do tipo de interação, pode ser visto abaixo.

Algoritmo 5: Grafo de Comportamento Mínimo

Entrada: Tree, Node

Saída: Status (Sucesso, Falha ou Executando)

```
1 Se (NodeType = Interator) então
2     Status ← Node→Iteration();
3 Se (Status = Sucesso) então
4     Tree→index ← Node→st;
5 Se Não Se Status = Falha então
6     Tree→index ← Node→ft;
7 Retorne Status;
```

3.5.1 Princípios da conversão da Árvore de Comportamento para Grafo de comportamento mínimo

1. **Preservação dos nós**: Para cada nó da árvore de comportamento do tipo *decorator* ou *interaction* original, existe um nó correspondente no grafo reduzido com a mesma função de execução ou modificação de resultado. Todos os nós originais da árvore do tipo *decorator* ou *interaction* são mantidos no grafo. Condições e ações continuam a ser blocos de funções com resultados que podem gerar duas transições (sucesso ou falha) ou permanência por mais um *tick*, e *decorators* funcionam como nós modificadores. Dessa forma, cada elemento da árvore tem seu equivalente direto no grafo.

2. **Transições baseadas em retorno**: A execução de cada nó leva, dependendo do resultado de sucesso ou falha, a um próximo nó definido de forma determinística. Na árvore, essa lógica fica implícita devido aos nós de controles aninhados. Para cada nó do tipo *decorator* ou *interaction*, a lógica booleana da árvore deve ser lida, sendo feita em dois passos, simulando sucesso e falha para o nó em questão, e verificando onde o mesmo para com base nas condições booleanas impostas a ele pela hierarquia.

3. **Equivalência de comportamento**: A ordem dos nós executados no grafo é a mesma que seria observada na árvore original, respeitando todas as regras de execução e *decorators*. Como cada transição no grafo reflete exatamente as condições da árvore, a execução segue os mesmos passos, pois a medida que os vértices vão sendo analisados, novas arestas vão sendo implementadas e marcadas. No final da execução, deve-se deletar todas as aretas e vértices não marcados ou inválidos, restando apenas os componentes válido, e sempre na ordem da esquerda para a direita. Com isso, a transformação preserva o comportamento final, apenas deixando o caminho de execução mais simples.

4. **Equivalência entre a árvore e o grafo**: É possível transformar qualquer árvore de comportamento com condições, ações e *decorators* em um grafo com transições condicionais, de forma que o comportamento de execução seja idêntico e ainda mais curto.

   Então temos que:

   - Todos os nós são preservados com suas funções originais;
   - As regras de execução se tornam transições claras no grafo de comportamento mínimo;
   - O caminho de execução no grafo é o mesmo da árvore.

   Essa transformação é feita de forma eficiente, proporcional ao tamanho da árvore, permitindo usar o grafo para análises, simulações ou verificações formais, com equivalência semântica completa, e propiciando qualquer sistema executar o mesmo comportamento com menos passos.

3.6 Reatividade

Em uma árvore de comportamento convencional, a estrutura consegue se readaptar a alterações nos nós anteriores. Isso é possível porque o *tick* é propagado a partir do nó *root*, passando principalmente pelos nós de controle. Esses nós de controle podem ser reativos ou não reativos. Os nós reativos reavaliam suas ramificações a cada tick, permitindo uma resposta dinâmica a mudanças no ambiente ou no estado do agente. Já os nós de controle simples, podem ter memória, ou seja, eles registram se já foram executados e seus estados. Isso é útil em casos como configuração de sistema, em que certas ações devem ocorrer apenas uma vez e não devem ser reexecutadas a cada tick.

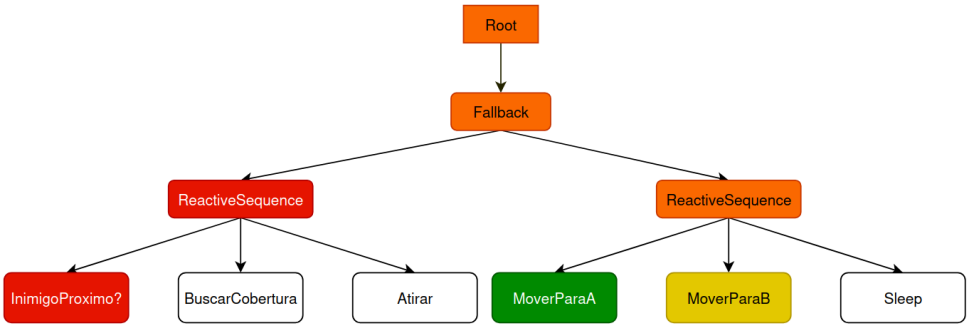


Figura 14: Nó MoverParaB em Execução.

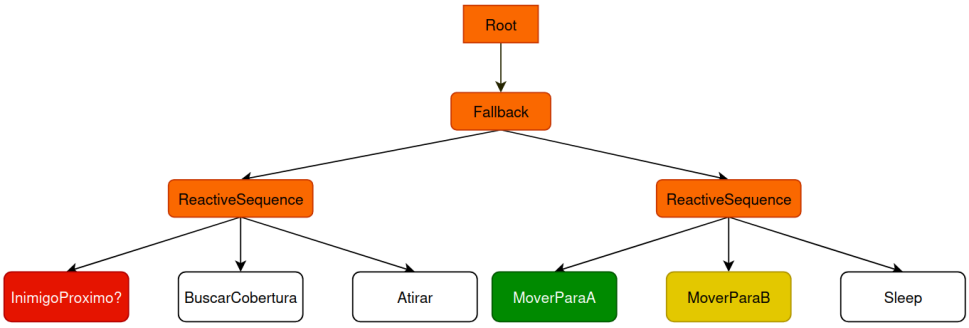


Figura 15: Tick no nó Reactive Sequence a esquerda.

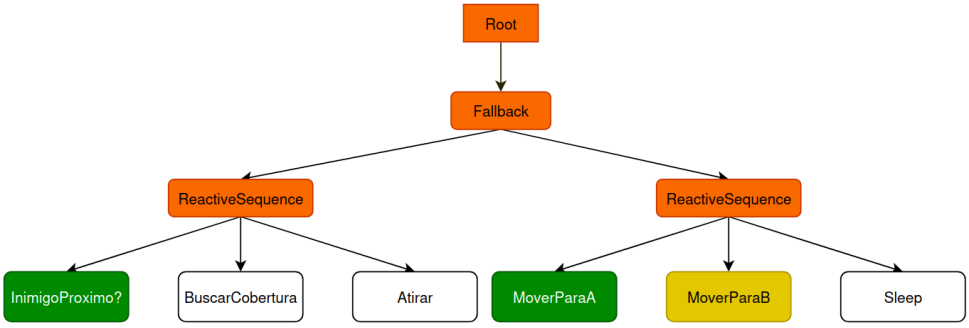


Figura 16: A condição *InimigoProximo?* é satisfeita e então a ramificação é continuada.

Legenda: Em execução Em espera Sucesso Erro

Fonte: Elaborado pelo autor, (2025)

Na figura 14, observamos que o nó em execução é o nó *MoverParaB*. No momento em que o *tick* é propagado em sua direção, ele parte inicialmente do nó *root*, passando pelo nó *fallback* e, em seguida, pelo nó *ReactiveSequence* à esquerda. Mesmo que esse nó tenha retornado falha anteriormente, como ilustrado na figura 15, ele é reexecutado, incluindo seus nós folhas. Já na figura 16, o nó de condição *InimigoProximo?* é satisfeito, o que provoca uma troca de ramificação. A memória dos nós pertencentes às outras ramificações pode ou não ser preservada, dependendo do tipo de reatividade implementado. Caso os nós não fossem reativos, a transição para a ramificação onde o nó *MoveParaB* está localizado, não seria totalmente validado, pois já estria salvo na sua memória.

Em um contexto embarcado, essa forma de execução pode ser bastante custosa, pois a cada *tick*, os nós anteriores podem ser reavaliados, o que compromete a estrutura mínima necessária. Uma abordagem interessante para a execução da árvore consiste em remover sua reatividade, mantendo, no entanto, seu comportamento funcional. Nessa abordagem, ao final da execução da árvore, apenas o nó de referência e o estado da árvore são reiniciados. Os dados já salvos na *blackboard* fica encarregada de realizar as checagens necessárias dos nós *fallbacks* e evitando comportamentos não necessários. Com isso, obtém-se um processamento mais simples, eficiente e adequado a sistemas embarcados com recursos limitados.

3.7 Testes e Validação

Para garantir o funcionamento correto do algoritmo, sua implementação e testes foram realizados nos *RTOSes MicriumOS* e *FreeRTOS*. Um *RTOS*, *Real-Time Operational System*, é "uma subclasse de sistema operacional destinado à concepção de sistemas computacionais, geralmente embarcados, em que o tempo de resposta a um evento é fixo e deve ser respeitado sempre que possível"(DENARDIN; BARRIQUELLO, 2019). Além disso, os testes serão conduzidos em duas placas diferentes, o *DevKit EFR32xG24-DK2601B* e o *DevKit E-Paper ESP32 Driver Board*, com aplicações distintas: uma aplicação de teste baseada em mensagens *print*, executada em ambas as placas, e uma aplicação prática que requer credenciais do *Wi-Fi* via *Bluetooth* e realiza a conexão com a rede.

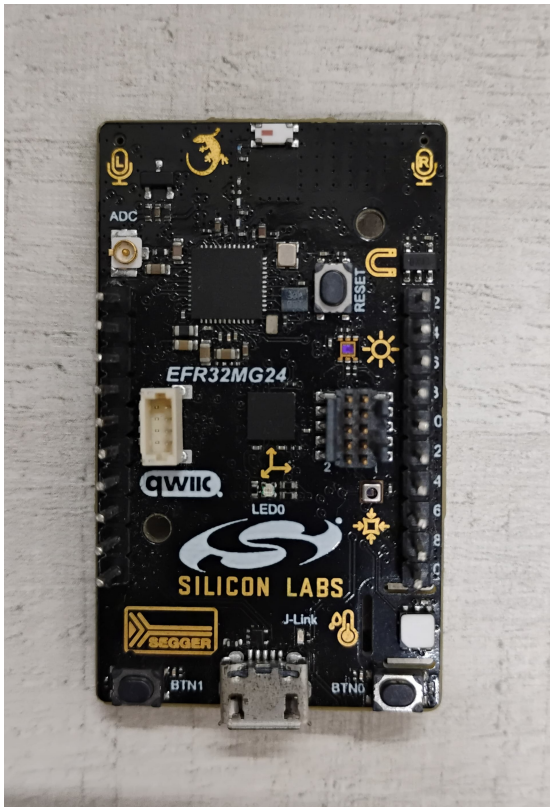


Figura 17: *DevKit EFR32xG24-DK2601B*.

Fonte: Elaborado pelo autor, (2025)

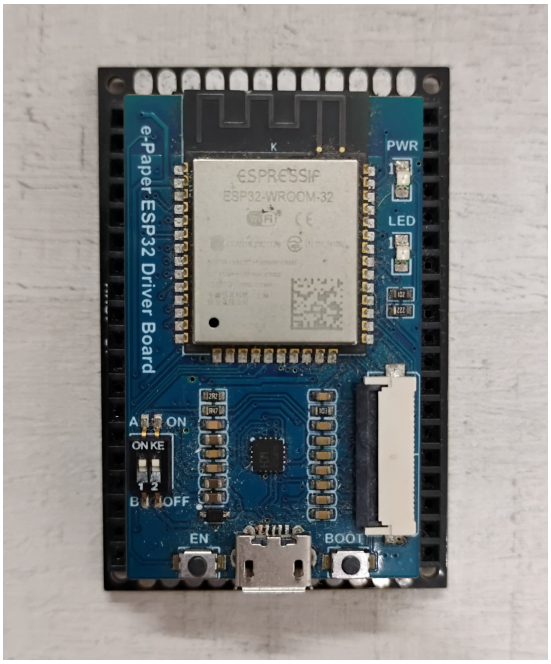


Figura 18: *DevKit E-Paper ESP32* Driver Board.

Fonte: Elaborado pelo autor, (2025)

A dependência do *firmware* em relação a plataforma ou ao *RTOS* é praticamente nula. No entanto, apenas uma função pode exigir uma chamada específica ao sistema operacional ou ao ambiente onde o código será executado: a rotina de *delay* (ou *sleep*). Essa função está localizada no arquivo ***btree_common.c***, onde os cálculos necessários para o tempo de espera — em horas, minutos, segundos e milissegundos — já são realizados.

Para desenvolvimento nesses DevKits, foram usados as *IDEs SimplicityIDE* e *Visual Code Studio*. O *SimplicityIDE* é um ambiente de desenvolvimento para dispositivos da Silicon Labs, fornecendo SDK (*Software Development Kit*) que fornece suporte a tecnologias da empresa, além de ter suporte ao *MicriumOS*. Já no *Visual Code Studio*, foi utilizado o *framework* da *ESP-IDF* para programar e configurar o *DevKit ESP32*. Esse ambiente permitiu o desenvolvimento do firmware, assim como upload direto para o dispositivo e testes.

4

Resultados e discussões

4.1 Suporte embarcado

O código da solução desenvolvida pode ser encontrada no apêndice A.

A estrutura embarcada implementada foi projetada para se adaptar de acordo ao tamanho total da árvore definida no projeto. O tamanho dessa árvore é determinado pela soma total de todos os nós utilizados dos grupo *Interators* e os nós *Decorators*. Os nós que possuem suporte são:

- ***Interators***: Representam estruturas simples, podendo conter apenas um ponteiro de função ou um valor numérico correspondente a um tempo de espera. São sempre nós folha:
  - ***Condition***: Nó folha vazia para implementar a condução;
  - ***Action***: Nó folha vazia para implementar a ação;
  - ***Sleep***: Nó folha que recebe um valor em milisegundos.
- ***Decorators***: Essas estruturas contêm um índice limite para limpeza de estados internos, um contador de execuções e um índice associado ao número de tentativas:
  - ***Force Success***;
  - **Force Fail**;
  - ***Inverter***;
  - ***Retry Until Success***: Recebe um valor máximo de tentativas;
  - ***Repeat***: Repete um valor de repetições;
  - ***Keep Running Until Success***.

O dimensionamento da árvore é realizado com base na quantidade total de nós definidos conforme os grupos acima. A estrutura de cada nó ocupa:

- **12 bytes**, se a quantidade total de nós for até 255;
- **16 bytes**, se o total for até 65.535;
- **20 bytes**, se a quantidade atingir até 4.294.967.295 nós.

Já a única estrutura que gerencia toda a árvore comportamental possuem os seguintes tamanhos:

- **16 bytes**, para árvores com até 255 nós;
- **20 bytes**, para árvores com até 65.535 nós;
- **28 bytes**, para árvores com até 4.294.967.295 nós.

Essa estratégia visa otimizar o uso de memória em sistemas embarcados, ajustando o tamanho das estruturas de dados conforme o tamanho da árvore implementada. Essas estruturas, por padrão, são geradas por *scripts* já para serem usadas na memória *flash*, do dispositivo, para ocupar a menor quantidade de *RAM* possível.

Uma estrutura de apoio também é definida para ser usada em uma função simples de validação da árvore. Essa estrutura possui quatro ponteiros para funções: um deles indica o tipo de execução (reativa ou não), e os outros três devem ser implementados pelo usuário, sendo responsáveis por tratar os casos em que a árvore está em *running*, *success* ou *fail*. Seu tamanho total é de 16 bytes.

4.2 Interações

Os nós de interação devem ser implementados individualmente. A geração inicial desses nós inclui apenas comentários básicos no arquivo `.h` com suas declarações e uma função vazia com um retorno de sucesso no arquivo `.c`.

Já o nó de `sleep` é implementado no arquivo `btree/src/btree_common.c`. Sua implementação depende da plataforma utilizada, cabendo ao usuário definir a função de pausa apropriada. Esse nó pode ser útil para simplificar a implementação em casos que envolvem múltiplas rotinas de espera.

4.3 Blackboard

A blackboard é uma estrutura estática em um arquivo `.c`, que guarda as variáveis que serão usadas. No arquivo `.h`, são definidos dois `enums`, um para buffers e um para valores, que são usados para acessar e controlar esses valores na balckboard, permitindo que o usuário possa implementar algum método de controle se houver necessidade para tal. Para a estrutura requisitar

esses dados, pode ser feito via quatro funções de suporte, a qual o usuário deve complementar a medida que incrementa dados na blackboard:

- ***Set Value***: Guarda um valor, com base no ID do valor;
- ***Get Value***: Requisita um valor, com base no ID do valor;
- ***Set Buffer***: Guarda um buffer, com base no ID do buffer;
- ***Get Buffer***: Requisita um buffer, com base no ID do buffer;

Conta também com uma função para limpar toda a *blackboard*, em caso de ***Error*** ou ***Fail*** da estrutura.

No arquivo `.h`, também é externalizada a estrutura diretamente, para acesso facilitado em toda a estrutura. Note que esse acesso pode causar problema de *Race Condition*, que é quando uma variável é alterada por multiplas *threads* simultâneamente.

4.4 Estados

Quanto aos estados que um nó ou a árvore podem assumir, temos:

- ***Running***: Indica que a árvore ou o nó está atualmente em execução.
- ***Success***: Indica que a árvore concluiu todo o comportamento com sucesso, ou que um nó completou seu comportamento com êxito.
- ***Fail***: Indica que a árvore finalizou com alguma falha no comportamento, ou que um nó não conseguiu executar seu comportamento corretamente.
- ***ReExecute***: Estado exclusivo de nós reativos, indicando a necessidade de reexecutar o mesmo caso receba um *tick*.
- ***StandBy***: Estado semelhante ao *Running*, mas utilizado para representar que o nó está aguardando ativação ou execução efetiva.
- ***Error***: Indica que houve um erro de formatação ou configuração incorreta na estrutura da árvore.

Quanto aos estados que podem ser armazenados na memória de cada nó, são utilizados apenas dois bits, totalizando quatro estados possíveis:

- ***Idle***: Indica que o nó ainda não foi iniciado. Seu valor é representado pelo binário `0b00`.
- ***Success***: Indica que o nó foi concluído com êxito. Seu valor é representado pelo binário `0b01`.

- ***Fail***: Indica que o nó falhou. Seu valor é representado pelo binário `0b10`.
- ***ReExecute***: Indica que o nó pode ser reexecutado, sendo usado principalmente por nós reativos. Seu valor é representado pelo binário `0b11`.

4.5 Scripts de Pré-processamento

Para converter a árvore gerada pela ferramenta Groot2, contida no arquivo XML, para o código final do sistema embarcado, foi desenvolvido um conjunto de scripts para o processamento. Esses scripts foram implementados na linguagem Python, escolhida por sua simplicidade, legibilidade e muitas bibliotecas para manipulação de arquivos e estruturas hierárquicas, principalmente em arquivos XML. O ambiente Python permitiu acelerar o desenvolvimento, facilitar testes e organizar o processo de geração automática de código. No total, foram desenvolvidos quatro scripts:

- `btree_init.py`: Script principal que atua como ponto de entrada do processo. Ele é responsável por configurar os parâmetros da geração automática, como o caminho do arquivo XML de entrada, o diretório de saída dos arquivos gerados, o nome da árvore principal, o nome do projeto (utilizado como prefixo nos nomes de funções e arquivos, precedido de *btree*), além de informações para o cabeçalho dos arquivos gerados como nome do desenvolvedor, e-mail, versão do projeto e copyright (se houver).
- `btree_array_creator.py`: Responsável pela leitura do arquivo XML e pela conversão da árvore de comportamento em uma estrutura intermediária baseada em arrays, conforme descrito na Seção 3.4. Essa estrutura temporária representa a árvore de forma simplificada, facilitando o processamento posterior. Uma visualização simples de como é feita essa montagem desse *array*, pode ser vista da figura 19, com base na árvore da figura 32.
- `btree_execute.py`: Processa o array intermediário gerado pelo script anterior e o converte em uma estrutura mínima otimizada, adequada para execução em sistemas embarcados, preservando a lógica e o comportamento da árvore original.
- `btree_creator.py`: Utiliza os dados processados para gerar os arquivos-fonte em linguagem C, com extensão `.c` e `.h`. Estes contém os arquivos do projeto, contendo o array das estruturas da árvore, itens comuns para a árvore (tamanho, ponteiro de tentativas), blackboard e os arquivos das funções dispostas nas árvores.

```
[0, 'BTREE_DEFINITION_CREATE_NODE_REACTIVE_FORCE_SUCCESS', 2, 'BTREE_DEFINITION_TREE_UNRELATED']
[1, 'BTREE_DEFINITION_CREATE_NODE_REACTIVE_CONDITION', 0, 0, '&btree_subtree_subtree_condition1']
[2, 'BTREE_DEFINITION_CREATE_NODE_REACTIVE_INVERTER', 4, 'BTREE_DEFINITION_TREE_UNRELATED']
[3, 'BTREE_DEFINITION_CREATE_NODE_REACTIVE_ACTION', 2, 2, '&btree_subtree_subtree_action1']
[4, 'BTREE_DEFINITION_CREATE_NODE_FORCE_FAIL', 7, 6]
[5, 'BTREE_DEFINITION_CREATE_NODE_CONDITION', 4, 4, '&btree_subtree_subtree_condition2']
[6, 'BTREE_DEFINITION_CREATE_NODE_ACTION', 7, 'BTREE_DEFINITION_TREE_UNRELATED', '&btree_subtree_subtree_action2']
[7, 'BTREE_DEFINITION_CREATE_NODE_REACTIVE_RETRY_UNTIL_SUCCESS', 11, 'BTREE_DEFINITION_TREE_UNRELATED', '5', 0, 10]
[8, 'BTREE_DEFINITION_CREATE_NODE_REACTIVE_CONDITION', 7, 9, '&btree_subtree_subtree_condition3']
[9, 'BTREE_DEFINITION_CREATE_NODE_REACTIVE_REPEAT', 7, 7, '5', 1, 10]
[10, 'BTREE_DEFINITION_CREATE_NODE_REACTIVE_ACTION', 9, 9, '&btree_subtree_subtree_action3']
[11, 'BTREE_DEFINITION_CREATE_NODE_KEEP_RUNNING_UNTIL_FAILURE', 13, 'BTREE_DEFINITION_TREE_UNRELATED', 12]
[12, 'BTREE_DEFINITION_CREATE_NODE_ACTION', 11, 11, '&btree_subtree_subtree_action4']
[13, 'BTREE_DEFINITION_CREATE_NODE_RETRY_UNTIL_SUCCESS', 'BTREE_DEFINITION_TREE_UNRELATED', 'BTREE_DEFINITION_TREE_UNRELATED', '9', 0, 15]
[14, 'BTREE_DEFINITION_CREATE_NODE_REACTIVE_ACTION', 15, 13, '&btree_subtree_subtree_action5']
[15, 'BTREE_DEFINITION_CREATE_NODE_REACTIVE_ACTION', 13, 13, '&btree_subtree_subtree_action6']
```

Figura 19: Array intermediário de criação.

Fonte: Elaborado pelo autor, (2025)

A execução do script `btree_init.py` é suficiente para iniciar e concluir todo o processo, pois ele executa internamente a execução dos demais scripts, resultando na geração completa do código-fonte embarcado a partir do XML exportado pelo Groot2. Esses scripts também geram os *buffers* usados pelos nós, atribuindo os endereços corretos para evitar compartilhamento de um mesmo local de memória de estado (nós em geral) ou de tentativas (especificamente nós *decorators*). Os tamanhos são definidos com base no tamanho da estrutura, onde o tamanho do *array* de estruturas é dividido por 16 e arredondado para cima, para garantir que cada nó tenha um espaço de 2 bits reservado de forma ordenada, e para os nós *decorators*, para evitar erros com base em estruturas reativas, deve-se considerar uma posição diferente para cada tipo de nó do tipo *Repeat* ou *Retry Until Success*.

4.6 Processamento

O processamento oferece suporte ao método reativo e não reativo. O método reativo se assemelha a uma árvore de comportamento reativa, que verifica os nós anteriores se eles forem do reativos, executando-os novamente a cada *tick*. No método não reatino, não são considerados as possíveis mudanças que possa ocorrer em nós anteriores. Quando um nó é executado, ele define já qual será o próximo nó a ser executado no próximo *tick*.

Em árvores não reativas, a execução do seu nó é direta, competindo aos nós de execuções uma chamada de função, retornando *standby* ou *running* para o caso de ações demoradas, sucesso ou falha em execuções rápidas ou aplicações de políticas, para assim, seu processamento determinar o seu proximo nó destino.

Já em árvores reativas, o processamento é semelhante ao das árvores não reativas, porém com a adição de que todos os nós possuem memória para navegar entre os nós, pois alguns nós devem ser reexecutados sempre que receberem um *tick*, e outros somente uma vez, visto que na implementação padrão, os nós de controle fariam esse filtro para esses *ticks*. Como as estruturas por padrão são guardadas em flash, esses dados devem ser guardados em memória

RAM, compartilhando um buffer com todos os nós, do tipo `uint32`, onde cada estado de cada nó usa somente dois bits para ocupar menos espaço, e que o endereço da estrutura serve como parâmetro para saber o índice no buffer e a quantidade de deslocamentos que deve ser feitas para salvar o novo valor do estado do nó.

4.7 Gerenciamento

O gerenciamento da estrutura pode ser feita em dois cenários, não reativa ou reativa.

- **Não reativa:** A execução nesse cenário é direta. Apenas verificamos a validade da estrutura e executamos o nó correspondente, armazenando seu estado de execução.
- **Reativa:** A execução do nó é feita de forma reativa, iniciando-se a partir do nó zero em uma versão simplificada da árvore. A cada execução, o estado do nó é salvo. Caso o nó seja reativo, ele será reexecutado independentemente do estado armazenado. Já os nós não reativos seguem a execução respeitando seus estados, conforme descrito na seção Estados.

A estrutura da árvore reduzida é finalizada quando um determinado valor é alcançado, definido pela macro *BTREE_DEFINITION_TREE_UNRELATED*. Esse valor corresponde ao último possível do tipo utilizado, que pode ser *uint8_t*, *uint16_t* ou *uint32_t*.

Há também uma função de suporte que pode apontar para três funções distintas, as quais o usuário pode implementar: uma para o caso de a árvore ser concluída com sucesso (por exemplo, para reiniciar a estrutura e configurar seu estado para *Running*); outra para o caso de falha (como reiniciar a estrutura, limpar a *blackboard* e configurar o estado para *Running*); e uma terceira para o caso em que a árvore ainda esteja em execução (por exemplo, para requisitar um novo *tick* da estrutura). Embora isso exija uma pequena implementação por parte do usuário, oferece maior flexibilidade na aplicação, permitindo a escolha entre um sistema bloqueante ou não, além de possibilitar a criação de comportamentos mais sofisticados.

4.8 Consumo

Para quantificar o consumo da aplicação, foi utilizando o *Simplicity IDE*, no momento de *build* onde é mostrado informações úteis do projeto. Criando um projeto com o *SDK* da *Silicon Labs* em *C*, em quatro aplicações diferentes, uma sem a importação da biblioteca, uma com um nó simples, uma com nó reativo, e uma outra aplicação com trinta nós reativos, temos os seguintes parâmentos:

Seção	Tamanho (bytes) sem a biblioteca	Tamanho (bytes) um nó simples	Tamanho (bytes) um nó reativo	Tamanho (bytes) trinta nós reativos
.text	8204	9100	9268	9616
.stack	4096	4096	4096	4096
.data	128	144	144	144
.bss	32	36	36	40
.heap	2048	2048	2048	2048

Tabela 4.1: Tabela de seções de memória e seus tamanhos.

Comparando as quatro colunas da tabela, é notável que a seção *.text*, localizada na *flash* do dispositivo, cresce 896 bytes quando importado a biblioteca de execução mais simples do grafo de comportamento mínimo, e quando importamos a reatividade, cresce 1064 bytes. Quando modificado a estrutura para trinta nós, o crescimento da seção *.text* se dá em 348 bytes em relação a coluna de um nó reativo, pois $12 * (30 - 1) = 348$. Agora em RAM, o acréscimo da importação da biblioteca é de 20 bytes, para um nó independente de ser uma execução simples ou reativa. Para trinta nós, o acréscimo total é de 24 bytes, comparado com a aplicação sem a importação da biblioteca, onde esse acréscimo de 4 bytes é devido à memória dos nós criados.

4.9 Resultados

Foram realizados três testes para avaliar o funcionamento da da estrutura, dois testes reativos apenas de *logs* usando o DevKit EFR32xG24-DK2601B e outro teste usando o *bluetooth* e o *Wi-Fi* do DevKit E-Paper ESP32.

4.9.1 Primeiro caso: Teste de reatividade

Considerando um teste básico apenas mostrando logs das funções, foi realizado um teste simples reativo de duas estruturas mínimas isoladas e depois com ambas juntas, para validar o funcionamento de multiplas estruturas simultâneas.

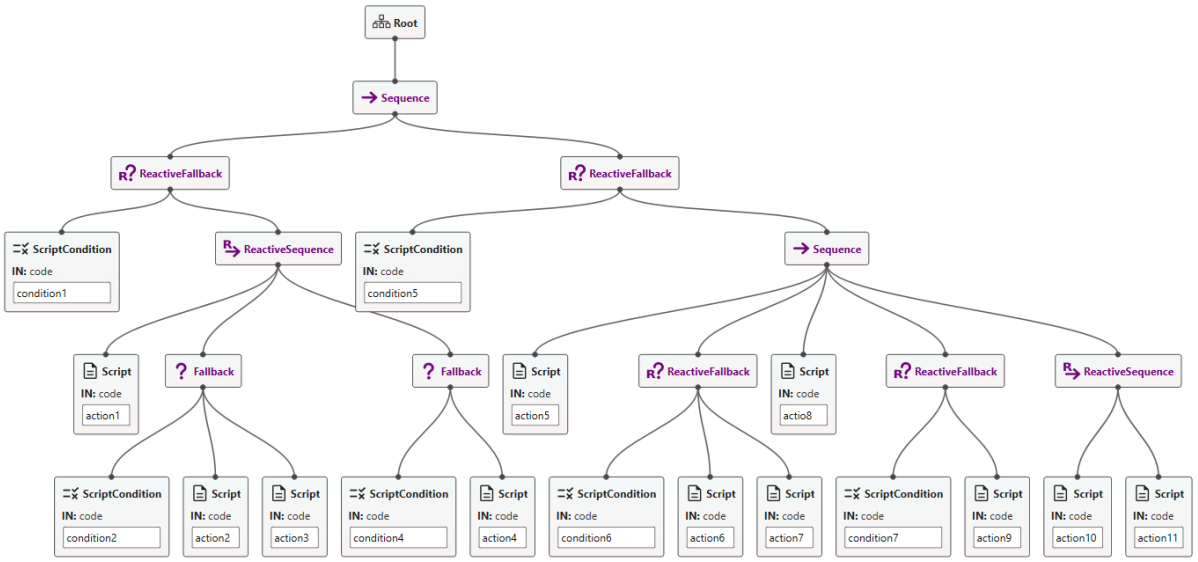


Figura 20: Árvore reativa de exemplo.

Fonte: Elaborado pelo autor, (2025)

Essa árvore não tem um propósito específico de aplicação, apenas para logs, para mostrar visualmente como é o comportamento da estrutura mínima reativa. As condições de sucesso para cada nó folha, são:

- Condition1: Precisa ser executado pelo menos 20 vezes para sucesso. Caso contrário, retorna falha;
- Action1: Sempre sucesso;
- Condition2: Ação3 deve ser executada pelo menos uma vez para retornar sucesso. Caso contrário, retorna falha;
- Action2: Sempre falha;
- Action2: Sempre sucesso;
- Condition4: Ação4 deve ser executada pelo menos uma vez para retornar sucesso. Caso contrário, retorna falha;
- Action4: Sempre sucesso;
- Condition5: Sempre falha;
- Action5: Sempre sucesso;
- Condition6: Ação6 deve ser executada pelo menos uma vez para retornar sucesso. Caso contrário, retorna falha;
- Action6: Sempre falha;
- Action7: Sempre sucesso;
- Action8: Sempre sucesso;
- Condition7: Sempre sucesso;
- Action9: Sempre sucesso;
- Action10: Sempre sucesso;

- Action11: Sempre sucesso;

```
const btree_definition_node_t btree_example_array[BTREE_EXAMPLE_ARRAY_SIZE] =
{
        [0] = BTREE_DEFINITION_CREATE_NODE_REACTIVE_CONDITION(7, 1, &btree_example_example_condition1), \
        [1] = BTREE_DEFINITION_CREATE_NODE_REACTIVE_ACTION(2, BTREE_DEFINITION_TREE_UNRELATED, &btree_example_example_action1), \
        [2] = BTREE_DEFINITION_CREATE_NODE_CONDITION(5, 3, &btree_example_example_condition2), \
        [3] = BTREE_DEFINITION_CREATE_NODE_ACTION(5, 4, &btree_example_example_action2), \
        [4] = BTREE_DEFINITION_CREATE_NODE_ACTION(5, BTREE_DEFINITION_TREE_UNRELATED, &btree_example_example_action3), \
        [5] = BTREE_DEFINITION_CREATE_NODE_CONDITION(7, 6, &btree_example_example_condition4), \
        [6] = BTREE_DEFINITION_CREATE_NODE_ACTION(7, BTREE_DEFINITION_TREE_UNRELATED, &btree_example_example_action4), \
        [7] = BTREE_DEFINITION_CREATE_NODE_REACTIVE_CONDITION(BTREE_DEFINITION_TREE_UNRELATED, 8, &btree_example_example_condition5), \
        [8] = BTREE_DEFINITION_CREATE_NODE_ACTION(9, BTREE_DEFINITION_TREE_UNRELATED, &btree_example_example_action5), \
        [9] = BTREE_DEFINITION_CREATE_NODE_REACTIVE_CONDITION(12, 10, &btree_example_example_condition6), \
        [10] = BTREE_DEFINITION_CREATE_NODE_REACTIVE_ACTION(12, 11, &btree_example_example_action6), \
        [11] = BTREE_DEFINITION_CREATE_NODE_REACTIVE_ACTION(12, BTREE_DEFINITION_TREE_UNRELATED, &btree_example_example_action7), \
        [12] = BTREE_DEFINITION_CREATE_NODE_ACTION(13, BTREE_DEFINITION_TREE_UNRELATED, &btree_example_example_acti08), \
        [13] = BTREE_DEFINITION_CREATE_NODE_REACTIVE_CONDITION(15, 14, &btree_example_example_condition7), \
        [14] = BTREE_DEFINITION_CREATE_NODE_REACTIVE_ACTION(15, BTREE_DEFINITION_TREE_UNRELATED, &btree_example_example_action9), \
        [15] = BTREE_DEFINITION_CREATE_NODE_REACTIVE_ACTION(16, BTREE_DEFINITION_TREE_UNRELATED, &btree_example_example_action10), \
        [16] = BTREE_DEFINITION_CREATE_NODE_REACTIVE_ACTION(BTREE_DEFINITION_TREE_UNRELATED, BTREE_DEFINITION_TREE_UNRELATED, &btree_example_example_action11), \
};
```

Figura 21: Array da estrutura mínima da árvore de comportamento Exemplo.

Fonte: Elaborado pelo autor, (2025)

A figura 21 mostra o array de estruturas da estrutura embarcada usada, enumerando os endereços que serão usados como referência para *logs*.

```
00> [0]c1 F
00> [0]c1 F [1]a1 S
00> [0]c1 F [1]a1 S [2]c2 F
00> [0]c1 F [1]a1 S [3]a2 F
00> [0]c1 F [1]a1 S [4]a3 S
00> [0]c1 F [1]a1 S [5]c4 F
00> [0]c1 F [1]a1 S [6]a4 S
00> [0]c1 F [1]a1 S [7]c5 F
00> [0]c1 F [1]a1 S [7]c5 F [8]a5 S
00> [0]c1 F [1]a1 S [7]c5 F [9]c6 F
00> [0]c1 F [1]a1 S [7]c5 F [9]c6 F [10]a6 F
00> [0]c1 F [1]a1 S [7]c5 F [9]c6 S [12]a8 S
00> [0]c1 F [1]a1 S [7]c5 F [9]c6 S [13]c7 S
00> [0]c1 F [1]a1 S [7]c5 F [9]c6 S [13]c7 S [15]a10 S
00> [0]c1 F [1]a1 S [7]c5 F [9]c6 S [13]c7 S [15]a10 S [16]a11 S success
00>
00> [0]c1 F
00> [0]c1 F [1]a1 S
00> [0]c1 F [1]a1 S [2]c2 S
00> [0]c1 F [1]a1 S [5]c4 S
00> [0]c1 F [1]a1 S [7]c5 F
00> [0]c1 F [1]a1 S [7]c5 F [8]a5 S
00> [0]c1 S [7]c5 F [9]c6 S
00> [0]c1 S [7]c5 F [9]c6 S [12]a8 S
00> [0]c1 S [7]c5 F [9]c6 S [13]c7 S
00> [0]c1 S [7]c5 F [9]c6 S [13]c7 S [15]a10 S
00> [0]c1 S [7]c5 F [9]c6 S [13]c7 S [15]a10 S [16]a11 S success
```

Figura 22: Log da estrutura mínima reativa Exemplo.

Fonte: Elaborado pelo autor, (2025)

A figura 22 mostra os *logs* de uma árvore reativa simples, onde cada linha é um *tick*. Note que a cada *tick*, a estrutura retorna para o nó zero, e o último nó executado da linha, é de fato o nó que seria executado no próximo *tick*, se não houvesse uma validação dos nós anteriores. Na figura também mostra duas execuções da árvore. Note que no sétimo *tick* da estrutura,

condition1 é alterada para sucesso, e como ela agora é sucesso, e na árvore de comportamento seu nó "pai"é um *fallback*, ele não precisa executar a ramificação "irmã"do nó *sequence*, não executando mais a *action1*, por ser o único nó reativo da ramificação.

A título de comparação, podemos usar a ferramenta dos mesmos desenvolvedores do GRoot2, a *BehaviorTree.CPP* (2023), onde podemos simular essa mesma árvore de comportamento em *C++*. O seu teste foi simples, não considerando um teste reativo real, apenas para visualização dos passos:

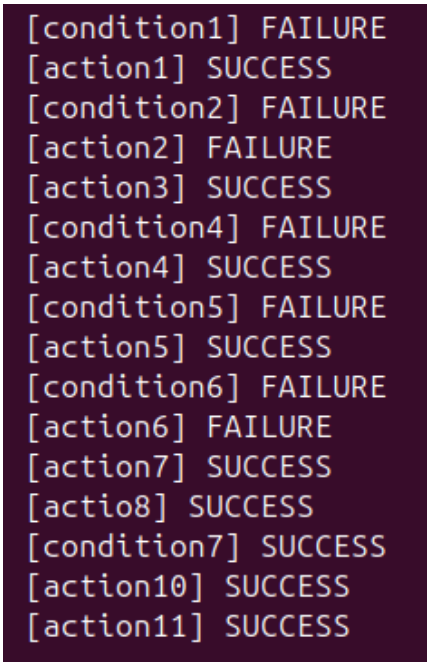


Figura 23: Saída da árvore de comportamento executada pela biblioteca *BehaviorTree.CPP*.
Fonte: Elaborado pelo autor, (2025)

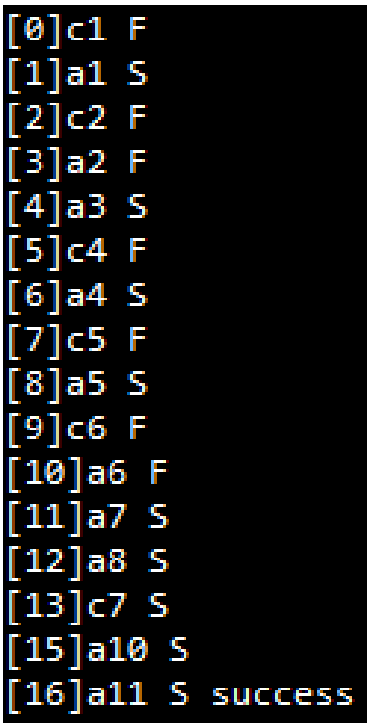


Figura 24: Saída da estrutura mínima sem reatividade.
Fonte: Elaborado pelo autor, (2025)

Pelas imagens 23 e 24, vemos que a saída é a mesma. Na imagem 24, tem todos os passos da execução da estrutura mínima implementada para sistemas embarcados. Agora veremos como é feito pela biblioteca de execução da *BehaviorTree.CPP*, para mostrar como é feito o passo a passo.

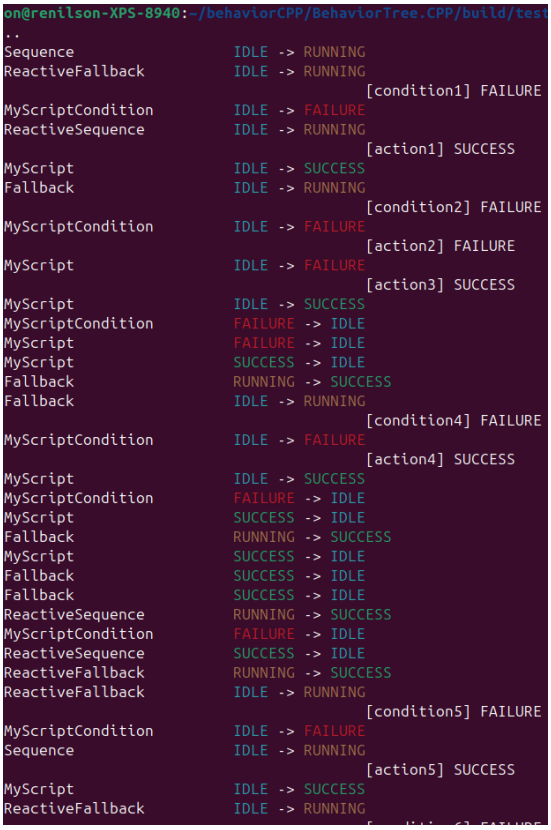


Figura 25: Árvore de comportamento simples sendo executada pelo biblioteca *BehaviorTree.CPP*, *log* 1.
Fonte: Elaborado pelo autor, (2025)

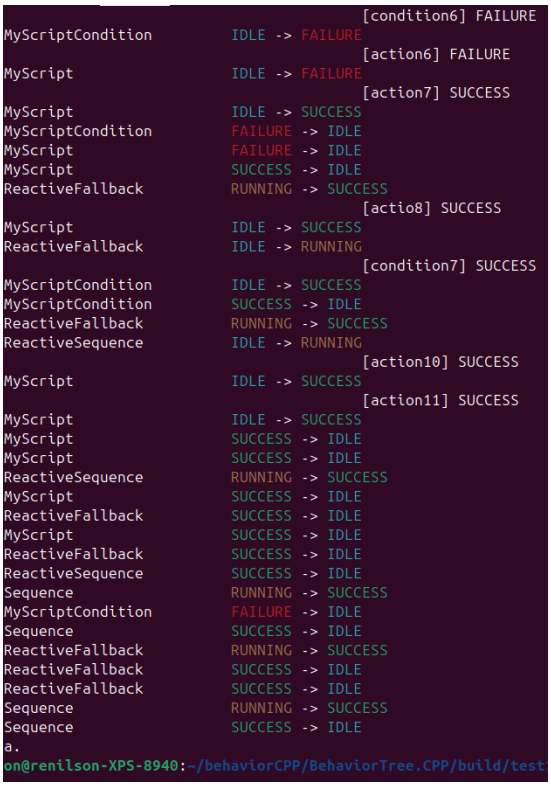


Figura 26: Árvore de comportamento simples sendo executada pelo biblioteca *BehaviorTree.CPP*, *log* 2.
Fonte: Elaborado pelo autor, (2025)

Comparando a execução feita pela biblioteca *BehaviorTree.CPP*, pode-se analisar por meio das imagens 25 e 26, que foram executados 31 vezes os nós de controles (somente considerado transições de estados dos respectivos nós) para a execução da árvore, enquanto a estrutura mínima navega por todo o grafo de forma direta, como visto no *log* da figura 24.

4.9.2 Segundo caso: Teste de mensagem

Esse teste consiste em criar uma árvore de comportamento grande o suficiente que trate 16 tipos de mensagens diferentes. Note que na figura 28 é apenas uma ramificação da árvore, onde a árvore possui 16 ramificações semelhantes a essa ao todo, tratando apenas uma condição diferente, mas tendo a mesma subárvore *process* igual.

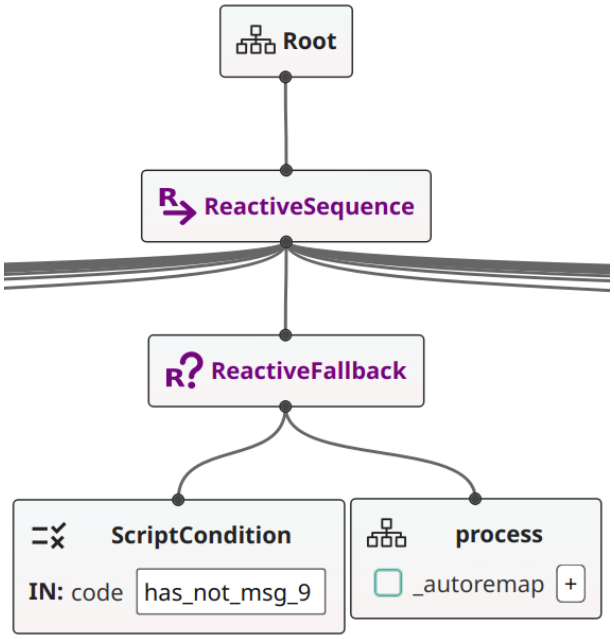


Figura 27: Uma das ramificações da árvore de comportamento *MSGS*.

Fonte: Elaborado pelo autor, (2025)

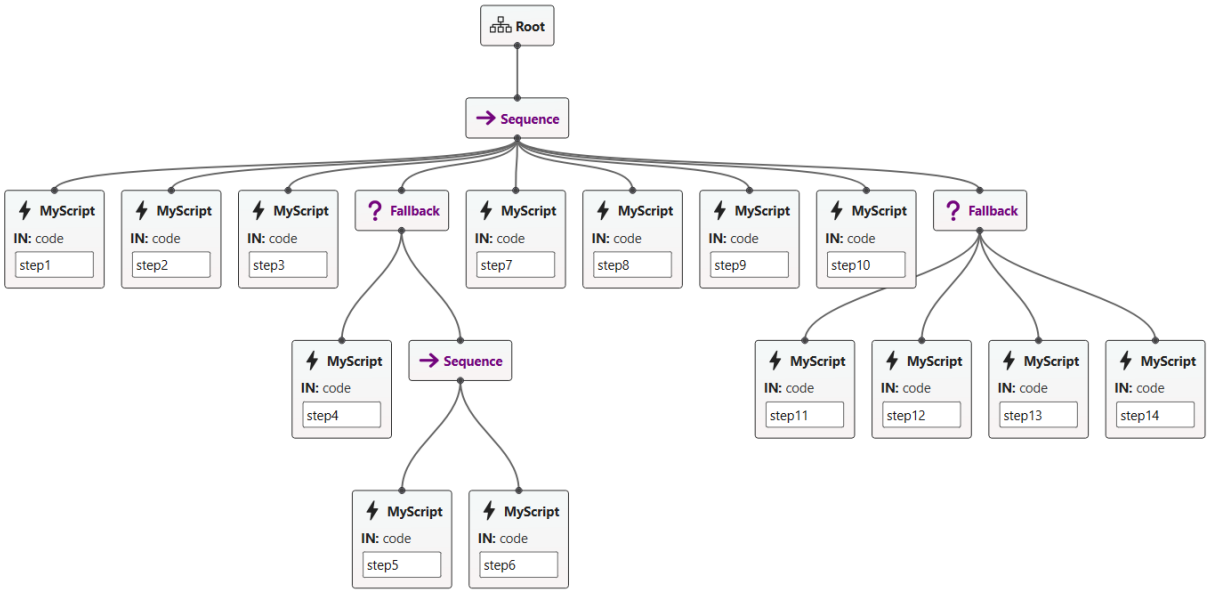


Figura 28: Subárvore *process*.

Fonte: Elaborado pelo autor, (2025)

Na árvore descrita na figura 28, os retornos das folhas são:

- has_not_msg_1: Sucesso;
- has_not_msg_2: Falha;
- has_not_msg_3: Sucesso;
- has_not_msg_4: Sucesso;
- has_not_msg_5: Falha;
- has_not_msg_6: Sucesso;
- has_not_msg_7: Falha;
- has_not_msg_8: Falha;
- has_not_msg_9: Falha;
- has_not_msg_10: Sucesso;
- has_not_msg_11: Falha;
- has_not_msg_12: Sucesso;
- has_not_msg_13: Falha;
- has_not_msg_14: Sucesso;
- has_not_msg_15: Falha;
- has_not_msg_16: Falha;
- Step1-14: Sempre sucesso;

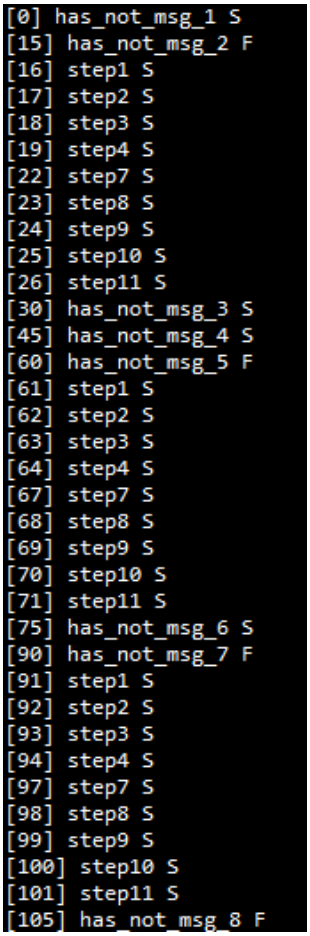


Figura 29: Saída da estrutura mínima sem reatividade, *log* 1.
Fonte: Elaborado pelo autor, (2025)

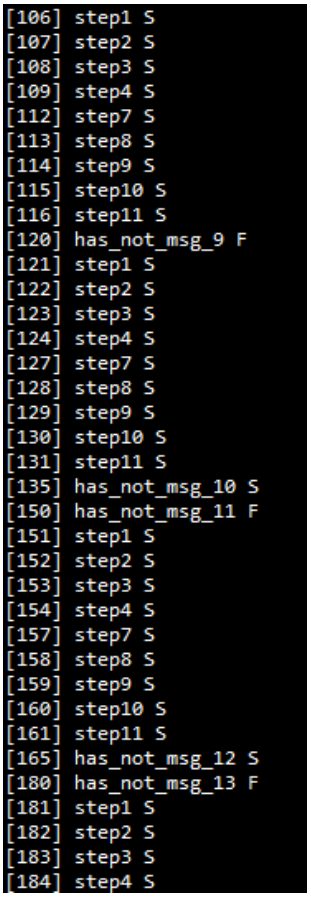


Figura 30: Saída da estrutura mínima sem reatividade, *log* 2.
Fonte: Elaborado pelo autor, (2025)

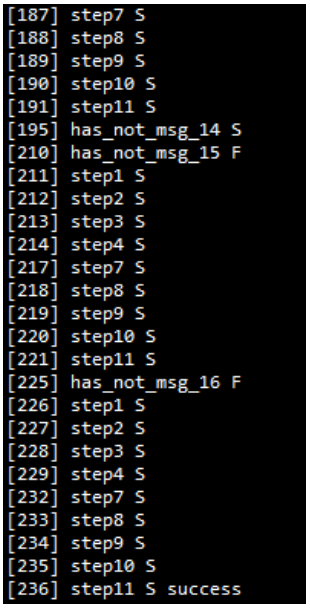


Figura 31: Saída da estrutura mínima sem reatividade, *log* 1.
Fonte: Elaborado pelo autor, (2025)

Em um teste simples com a *BehaviorTree.CPP*, desconsiderando a reatividade e focando apenas na quantificação dos nós de controle, foi registrado um total de 157 execuções desses nós. As figuras 29 à 31 mostram como fica a saída na estrutura mínima a árvore de comportamento retratada na 27.

4.9.3 Terceiro caso: Teste de reatividade com decorators

Semelhante ao exemplo anterior, esse exemplo busca últizar todos os nós suportados na solução embarcada.

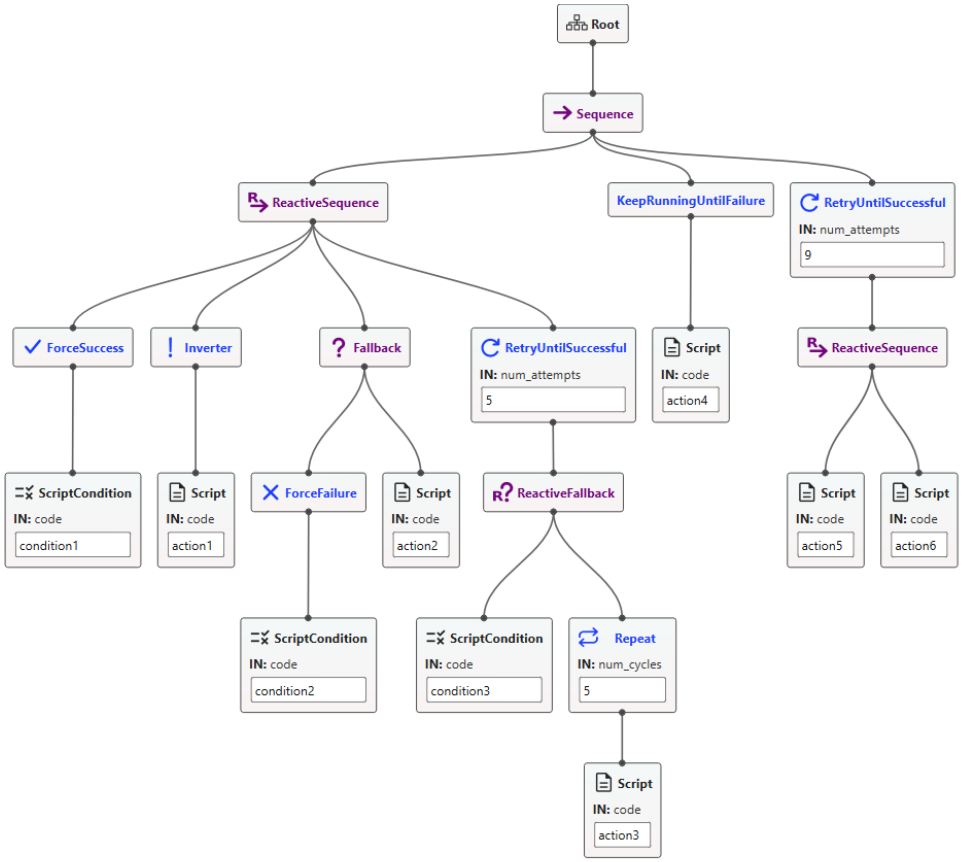


Figura 32: Árvore reativa da SubTree.

Fonte: Elaborado pelo autor, (2025)

As condições de sucesso para cada nó folha, são:

- Condition1: Precisa ser executado pelo menos 2 vezes para sucesso. Caso contrário, retorna falha;
- Action1: Sempre sucesso;
- Condition2: Sempre sucesso;
- Action2: Sempre sucesso;
- Condition3: Precisa ser executado pelo menos 5 vezes para sucesso. Caso contrário, retorna falha;
- Action3: Precisa ser executado pelo menos 5 vezes para sucesso. Caso contrário, retorna falha;
- Action4: Precisa ser executado pelo menos 3 vezes para sucesso. Caso contrário, retorna falha;

- Action5: Precisa ser executado pelo menos 2 vezes para sucesso. Caso contrário, retorna falha;
- Action6: Sempre sucesso;

```
const btree_definition_node_t btree_subtree_array[BTREE_SUBTREE_ARRAY_SIZE] =
{
        [0] = BTREE_DEFINITION_CREATE_NODE_REACTIVE_FORCE_SUCCESS(2, BTREE_DEFINITION_TREE_UNRELATED, 1), \
        [1] = BTREE_DEFINITION_CREATE_NODE_REACTIVE_CONDITION(0, 0, &btree_subtree_subtree_condition1), \
        [2] = BTREE_DEFINITION_CREATE_NODE_REACTIVE_INVERTER(4, BTREE_DEFINITION_TREE_UNRELATED, 3), \
        [3] = BTREE_DEFINITION_CREATE_NODE_REACTIVE_ACTION(2, 2, &btree_subtree_subtree_action1), \
        [4] = BTREE_DEFINITION_CREATE_NODE_FORCE_FAIL(7, 6, 5), \
        [5] = BTREE_DEFINITION_CREATE_NODE_CONDITION(4, 4, &btree_subtree_subtree_condition2), \
        [6] = BTREE_DEFINITION_CREATE_NODE_ACTION(7, BTREE_DEFINITION_TREE_UNRELATED, &btree_subtree_subtree_action2), \
        [7] = BTREE_DEFINITION_CREATE_NODE_REACTIVE_RETRY_UNTIL_SUCCESS(11, BTREE_DEFINITION_TREE_UNRELATED, 8, 5, 0, 10), \
        [8] = BTREE_DEFINITION_CREATE_NODE_REACTIVE_CONDITION(7, 9, &btree_subtree_subtree_condition3), \
        [9] = BTREE_DEFINITION_CREATE_NODE_REACTIVE_REPEAT(7, 7, 10, 5, 1, 10), \
        [10] = BTREE_DEFINITION_CREATE_NODE_REACTIVE_ACTION(9, 9, &btree_subtree_subtree_action3), \
        [11] = BTREE_DEFINITION_CREATE_NODE_KEEP_RUNNING_UNTIL_FAILURE(13, BTREE_DEFINITION_TREE_UNRELATED, 12, 12), \
        [12] = BTREE_DEFINITION_CREATE_NODE_ACTION(11, 11, &btree_subtree_subtree_action4), \
        [13] = BTREE_DEFINITION_CREATE_NODE_RETRY_UNTIL_SUCCESS(BTREE_DEFINITION_TREE_UNRELATED, BTREE_DEFINITION_TREE_UNRELATED, 14, 9, 2, 15), \
        [14] = BTREE_DEFINITION_CREATE_NODE_REACTIVE_ACTION(15, 13, &btree_subtree_subtree_action5), \
        [15] = BTREE_DEFINITION_CREATE_NODE_REACTIVE_ACTION(13, 13, &btree_subtree_subtree_action6), \
};
```

Figura 33: Array da estrutura mínima da árvore de comportamento SubTree.

Fonte: Elaborado pelo autor, (2025)

A figura 33 mostra o array de estruturas da estrutura embarcada usada, enumerando os endereços que serão usados como referência para *logs*.

```
00> [0]R
00> [1]c1 F
00> [0]S
00> [0]R [1]c1 F [0]S [2]R
00> [0]R [1]c1 F [0]S [3]a1 F
00> [0]R [1]c1 S [0]S [2]S
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [4]R
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [5]c2 S
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [4]F
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [6]a2 S
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [7]
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [8]c3 F
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [8]c3 F [9]
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [8]c3 F [10]a3 F
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [8]c3 F [9]R
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [8]c3 F [10]a3 F
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [8]c3 F [9]R
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [8]c3 S [7]S [11]
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [7][8]c3 S [7]S [12]a4 F
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [7][8]c3 S [7]S [11]
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [7][8]c3 S [7]S [12]a4 F
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [7][8]c3 S [7]S [11]
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [7][8]c3 S [7]S [12]a4 F
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [7][8]c3 S [7]S [11]
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [7][8]c3 S [7]S [12]a4 F
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [7][8]c3 S [7]S [11]
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [7][8]c3 S [7]S [12]a4 S
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [7][8]c3 S [7]S [11]S
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [7][8]c3 S [7]S [13]
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [7][8]c3 S [7]S [14]a5 F
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [7][8]c3 S [7]S [13]R
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [7][8]c3 S [7]S [14]a5 F
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [7][8]c3 S [7]S [13]R
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [7][8]c3 S [7]S [14]a5 F
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [7][8]c3 S [7]S [13]R
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [7][8]c3 S [7]S [14]a5 S
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [7][8]c3 S [7]S [14]a5 S [15]a6 S
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [7][8]c3 S [7]S [13]S success
00>
00> [0]R
00> [1]c1 S
00> [0]S
00> [0]R [1]c1 S [0]S [2]R
00> [0]R [1]c1 S [0]S [3]a1 F
00> [0]R [1]c1 S [0]S [2]S
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [4]F
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [6]a2 S
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [7]
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [8]c3 S
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [7]S
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [7][8]c3 S [7]S [11]S
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [7][8]c3 S [7]S [13]S success
```

Figura 34: Log da estrutura mínima reativa SubTree.

Fonte: Elaborado pelo autor, (2025)

A figura 34 mostra os *logs* de uma árvore reativa simples SubTree, onde cada linha é um *tick*. Note que a cada *tick* que um nó *decorator* mostra "R", informando que ainda está em execução, pois está esperando algo (note que em alguns casos, como o nó 11, ou o nó 7, não possuem nada, pois não foram colocados *logs* nesses cenários, mas são a inicialização dos nós de tentativas, que retornam *Running*).

Testando agora no *BehaviorTree.CPP*, deve-se retirar o nó *KeepRunningUntilSuccess*, pois a lógica do funcionamento é diferente, enquanto a ferramenta utiliza para executar exaustivamente uma ramificação enquanto ela for sucesso, na estrutura mínima é usado para continuar enquanto falha. Já o nó *Repeat*, finaliza sua execução se sua ramificação falhar na *BehaviorTree.CPP*, então nesse teste, foi modificada para sempre ter sucesso. A ação 5, foi alterada também para ser sucesso.

Com base na execução no *BehaviorTree.CPP*, e ainda comparando a quantidade de vezes que executamos e trocamos os estados dos nós do tipo *decorators*, temos:

Tabela 4.2: Comparação de chamadas de nós

Nó	**Execuções na *BehaviorTree.CPP***	**Execuções na Estrutura Mínima**	**Número do nó na Estrutura Mínima**
Fallback ou *Sequence*	15	0	—-
ForceSuccess	3	2	0
Inverter	3	2	1
ForceFail	2	2	4
RetryUntilSuccessful (esquerda)	3	2	7
Repeat	3	6	9
RetryUntilSuccessful (direita)	3	2	12

4.9.4 Quarto caso: Teste em conjunto

Esse teste é mais simples, onde ele básicamente inicia ambas estruturas em duas *tasks* diferentes, que atuam como *threads*.

```
00> [0]c1 F
00> [0]R
00> [0]c1 F [1]a1 S
00> [1]c1 F
00> [0]c1 F [1]a1 S [2]c2 F
00> [0]S
00> [0]c1 F [1]a1 S [3]a2 F
00> [0]R [1]c1 F [0]S [2]R
00> [0]c1 F [1]a1 S [4]a3 S
00> [0]R [1]c1 F [0]S [3]a1 F
00> [0]c1 F [1]a1 S [5]c4 F
00> [0]R [1]c1 S [0]S [2]S
00> [0]c1 F [1]a1 S [6]a4 S
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [4]R
00> [0]c1 F [1]a1 S [7]c5 F
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [5]c2 S
00> [0]c1 F [1]a1 S [7]c5 F [8]a5 S
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [4]F
00> [0]c1 F [1]a1 S [7]c5 F [9]c6 F
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [6]a2 S
00> [0]c1 F [1]a1 S [7]c5 F [9]c6 F [10]a6 F
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [7]
00> [0]c1 F [1]a1 S [7]c5 F [9]c6 S [12]a8 S
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [8]c3 F
00> [0]c1 F [1]a1 S [7]c5 F [9]c6 S [13]c7 S
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [8]c3 F [9]
00> [0]c1 F [1]a1 S [7]c5 F [9]c6 S [13]c7 S [15]a10 S
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [8]c3 F [10]a3 F
00> [0]c1 F [1]a1 S [7]c5 F [9]c6 S [13]c7 S [15]a10 S [16]a11 S success
00>
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [8]c3 F [9]R
00> [0]c1 F
00> [0]R [1]c1 S [0]
00> [0]c1 S [7]c5 F [9]c6 S [13]c7 S
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [7][8]c3 S [7]S [11]
00> [0]c1 S [7]c5 F [9]c6 S [13]c7 S [15]a10 S
00> [0]R [1]c1 S [0]S [2]R [3]a1 F [2]S [7][8]c3 S [7]S [12]a4 F
00> [0]c1 S [7]c5 F [9]c6 S [13]c7 S [15]a10 S [16]a11 S success
```

Figura 35: Logs das estruturas mínima reativa de Exemplo e SubTree.

Fonte: Elaborado pelo autor, (2025)

A figura 35 mostra que duas estruturas mínimas, conseguem ser executadas em paralelas, em *threads* ou *tasks* diferentes.

4.9.5 Quinto caso: Teste de aplicação Bluetooth e Wi-Fi

Considerando agora uma implementação embarcada, é possível realizar uma solução simples, porém eficiente, na qual a aplicação é responsável por iniciar o Bluetooth do DevKit ESP32, aguardar o usuário inserir as credenciais nas características Bluetooth *0xFF01* e *0xFF02*, que recebem, respectivamente, o *SSID* e a *senha* da rede *Wi-Fi* à qual se deseja conectar. Vale pontuar que o DevKit ESP32 utilizado, não oferece suporte simultâneo ao *bluetooth* e *Wi-Fi*. Note também que nesse exemplo não haverá reatividade, ou seja, não haverá validação dos nós anteriores, até a estrutura ser finalizada e reiniciada.

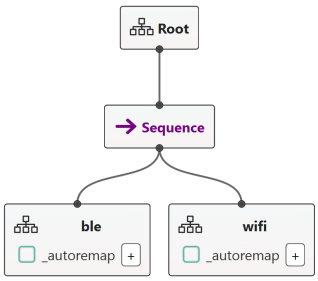


Figura 36: Árvore principal da aplicação bluetooth com Wi-Fi.

Fonte: Elaborado pelo autor, (2025)

Inicialmente, como podemos ver na figura 36, temos uma aplicação que simplesmente chama em sequencia o *bluetooth* e o *Wi-Fi*.

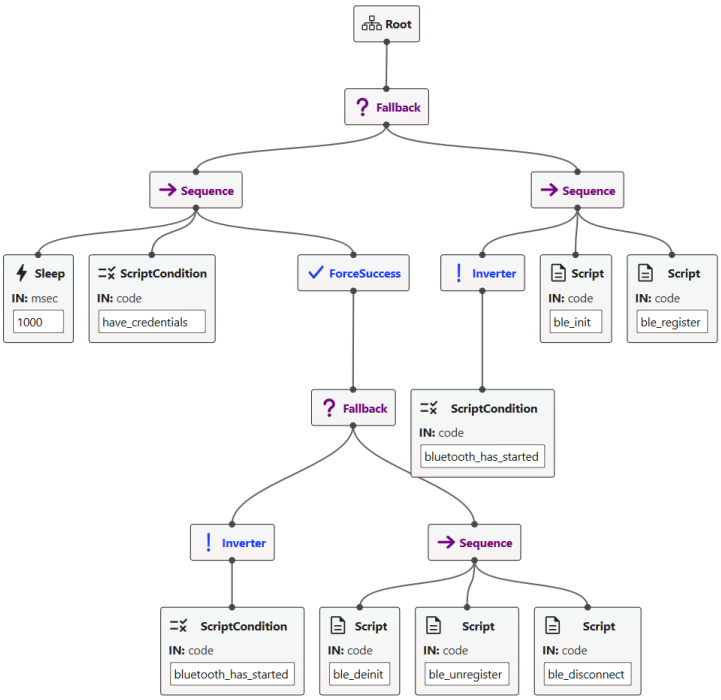


Figura 37: Subárvore bluetooth.

Fonte: Elaborado pelo autor, (2025)

Na subárvore responsável pelo Bluetooth, existem duas ramificações principais. A da esquerda inicia com um nó de espera (*sleep*), utilizado para garantir que as configurações iniciais do sistema, como a inicialização da *NVS* da ESP32, estejam concluídas. Em seguida, é verificado se existem credenciais válidas armazenadas na *blackboard*. Caso existam, entende-se que o Bluetooth já pode ser desligado. A próxima etapa consiste em verificar se o Bluetooth foi iniciado, por meio da condição *bluetooth_has_started*. Essa condição tem seu valor invertido: se o Bluetooth já estiver iniciado, a verificação retorna falha, indicando que devemos desabilitá-lo utilizando as ações *ble_deinit*, *ble_unregister* e *ble_disconnect*.

Caso o Bluetooth ainda não tenha sido iniciado, a árvore segue para a segunda ramificação. Nela, a condição *bluetooth_has_started* é verificada novamente, com o valor também invertido, devido ao comportamento do nó *sequence* acima. Se a condição for satisfeita, os nós *ble_init* e *ble_register* são executados, iniciando o módulo Bluetooth.

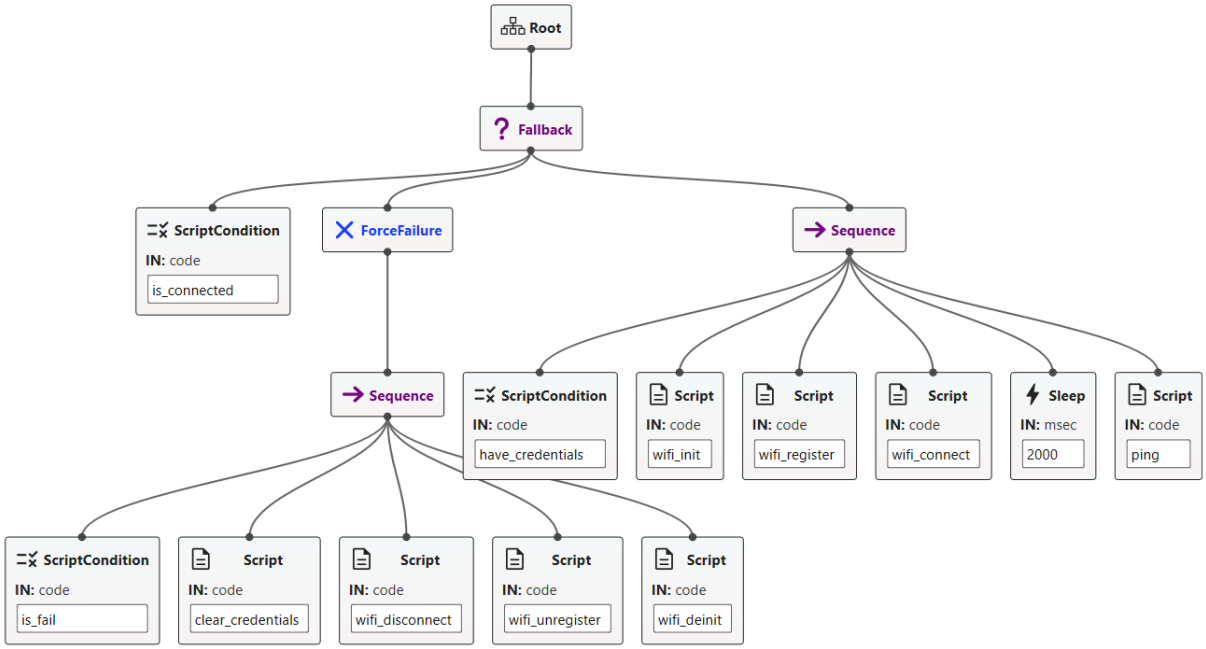


Figura 38: Subárvore Wi-Fi.

Fonte: Elaborado pelo autor, (2025)

Já na subárvore responsável pelo Wi-Fi, inicialmente é verificado se o dispositivo está conectado à rede. Caso não esteja, realiza-se uma verificação para saber se ele falhou previamente no *Wi-Fi* para assim iniciar uma rotina de desligamento do mesmo, e nessa ramificação sempre é falha pelo nó *decorator*. Na ramificação ao lado, é checada se há credenciais válidas, e em caso positivo, o módulo *Wi-Fi* é inicializado por meio dos nós *wifi_init*, *wifi_register* e *wifi_connect*, finalizando com um *ping* na rede, que retorna um *StandBy*, se conseguir fazer a soliciatação de envio de pacotes. Esse nó de *ping*, chama *callbacks* de sucesso e falha, caso não consiga compartilhar os pacotes com a internet, definindo um valor na *blackboard* por meio destes, e no início do nó de *ping*, ele deve verificar se esse valor foi alterado e retornar de acordo com o

valor definido pelo *callback* chamado.

Após essa etapa, é inserido um *sleep* de dois segundos, seguido de um novo *ping*, e então, aguarda ou o ping falhar, ou ter sucesso. Quando recebe essa informação do ping, é repassada uma *flag* para a tarefa principal que indica que deve realizar o tick. Caso a conexão ainda não tenha sido estabelecida, a verificação é repetida. Persistindo a falha, as credenciais são apagadas pela ação *clear_credentials*, e o Wi-Fi é completamente desativado, com a execução das ações *wifi_disconnect*, *wifi_unregister* e *wifi_deinit*.

```
const btree_definition_node_t btree_main_array[BTREE_MAIN_ARRAY_SIZE] =
{
        [0] = BTREE_DEFINITION_CREATE_NODE_ACTION_SLEEP(1, 8, 1000), \
        [1] = BTREE_DEFINITION_CREATE_NODE_CONDITION(2, 8, &btree_main_ble_have_credentials), \
        [2] = BTREE_DEFINITION_CREATE_NODE_FORCE_SUCCESS(12, 8), \
        [3] = BTREE_DEFINITION_CREATE_NODE_INVERTER(2, 5), \
        [4] = BTREE_DEFINITION_CREATE_NODE_CONDITION(3, 3, &btree_main_ble_bluetooth_has_started), \
        [5] = BTREE_DEFINITION_CREATE_NODE_ACTION(6, 2, &btree_main_ble_ble_deinit), \
        [6] = BTREE_DEFINITION_CREATE_NODE_ACTION(7, 2, &btree_main_ble_ble_unregister), \
        [7] = BTREE_DEFINITION_CREATE_NODE_ACTION(2, 2, &btree_main_ble_ble_disconnect), \
        [8] = BTREE_DEFINITION_CREATE_NODE_INVERTER(10, BTREE_DEFINITION_TREE_UNRELATED), \
        [9] = BTREE_DEFINITION_CREATE_NODE_CONDITION(8, 8, &btree_main_ble_bluetooth_has_started), \
        [10] = BTREE_DEFINITION_CREATE_NODE_ACTION(11, BTREE_DEFINITION_TREE_UNRELATED, &btree_main_ble_ble_init), \
        [11] = BTREE_DEFINITION_CREATE_NODE_ACTION(12, BTREE_DEFINITION_TREE_UNRELATED, &btree_main_ble_ble_register), \
        [12] = BTREE_DEFINITION_CREATE_NODE_CONDITION(BTREE_DEFINITION_TREE_UNRELATED, 13, &btree_main_wifi_is_connected), \
        [13] = BTREE_DEFINITION_CREATE_NODE_FORCE_FAIL(BTREE_DEFINITION_TREE_UNRELATED, 19), \
        [14] = BTREE_DEFINITION_CREATE_NODE_CONDITION(15, 13, &btree_main_wifi_is_fail), \
        [15] = BTREE_DEFINITION_CREATE_NODE_ACTION(16, 13, &btree_main_wifi_clear_credentials), \
        [16] = BTREE_DEFINITION_CREATE_NODE_ACTION(17, 13, &btree_main_wifi_wifi_disconnect), \
        [17] = BTREE_DEFINITION_CREATE_NODE_ACTION(18, 13, &btree_main_wifi_wifi_unregister), \
        [18] = BTREE_DEFINITION_CREATE_NODE_ACTION(13, 13, &btree_main_wifi_wifi_deinit), \
        [19] = BTREE_DEFINITION_CREATE_NODE_CONDITION(20, BTREE_DEFINITION_TREE_UNRELATED, &btree_main_ble_have_credentials), \
        [20] = BTREE_DEFINITION_CREATE_NODE_ACTION(21, BTREE_DEFINITION_TREE_UNRELATED, &btree_main_wifi_wifi_init), \
        [21] = BTREE_DEFINITION_CREATE_NODE_ACTION(22, BTREE_DEFINITION_TREE_UNRELATED, &btree_main_wifi_wifi_register), \
        [22] = BTREE_DEFINITION_CREATE_NODE_ACTION(23, BTREE_DEFINITION_TREE_UNRELATED, &btree_main_wifi_wifi_connect), \
        [23] = BTREE_DEFINITION_CREATE_NODE_ACTION_SLEEP(24, BTREE_DEFINITION_TREE_UNRELATED, 2000), \
        [24] = BTREE_DEFINITION_CREATE_NODE_ACTION(BTREE_DEFINITION_TREE_UNRELATED, BTREE_DEFINITION_TREE_UNRELATED, &btree_main_wifi_ping), \
};
```

Figura 39: Array da estrutura Bluetooth e Wi-Fi.

Fonte: Elaborado pelo autor, (2025)

A figura 39 mostra o array de estruturas da estrutura embarcada usada, enumerando os endereços que serão usados como referência para *logs*.

```
I (687) main_task: Started on CPU0
I (697) main_task: Calling app_main()
[0]S
[1]Credenciais ainda não recebidas.F
[8]R
[9]F
[8]S
[10]Initializing BLEI (6717) BTDM_INIT: BT controller compile version [dc1cd58]
I (6717) BTDM_INIT: Bluetooth MAC: 94:b5:55:1b:05:16
I (6717) phy_init: phy_version 4860,6b7a6e5,Feb  6 2025,14:47:07
S
[11]Registering BLE callbacksESP_GATTS_REG_EVTS
Advertising data set complete, starting advertising...Advertising started[12]F
[13]R
Device connected[14]F
[13]F
[19]Credenciais ainda não recebidas.F fail
```

Figura 40: Primeiro log da aplicação BLE + Wi-Fi.

Fonte: Elaborado pelo autor, (2025)

A figura 40 mostra os *logs* do framework da ESP32 referentes ao *boot* da ESP32, início da aplicação *bluetooth* e primeiro loop falho da árvore, devido a falha de conexão.

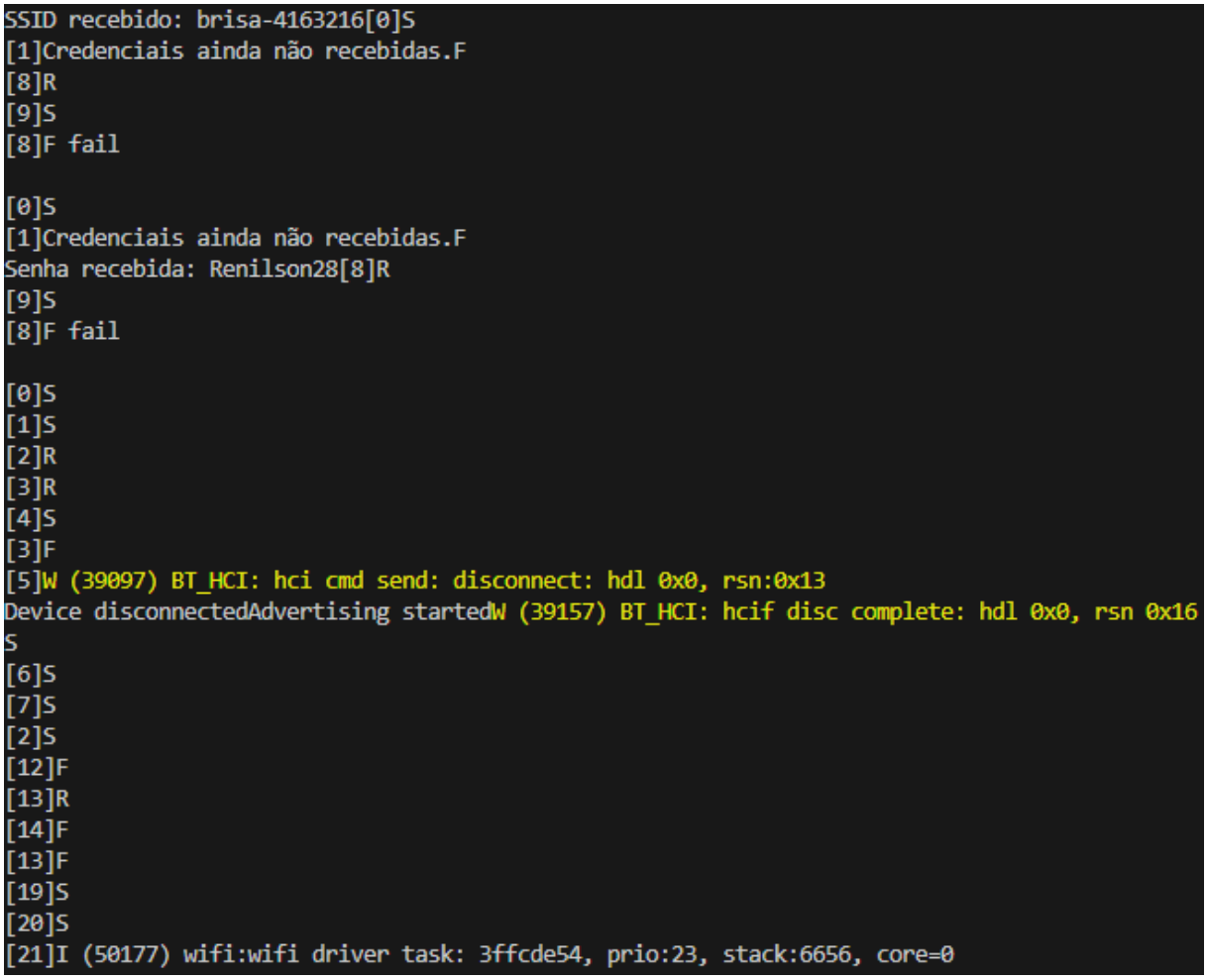


Figura 41: Segundo log da aplicação BLE + Wi-Fi.

Fonte: Elaborado pelo autor, (2025)

A figura 41 mostra os *logs* do framework da ESP32 referentes ao fechamento da conexão *bluetooth*, pois foi recebido a *senha* e o *SSID* da rede *Wi-Fi*. Vemos também o loop da aplicação, sendo periódica, buscando validação.

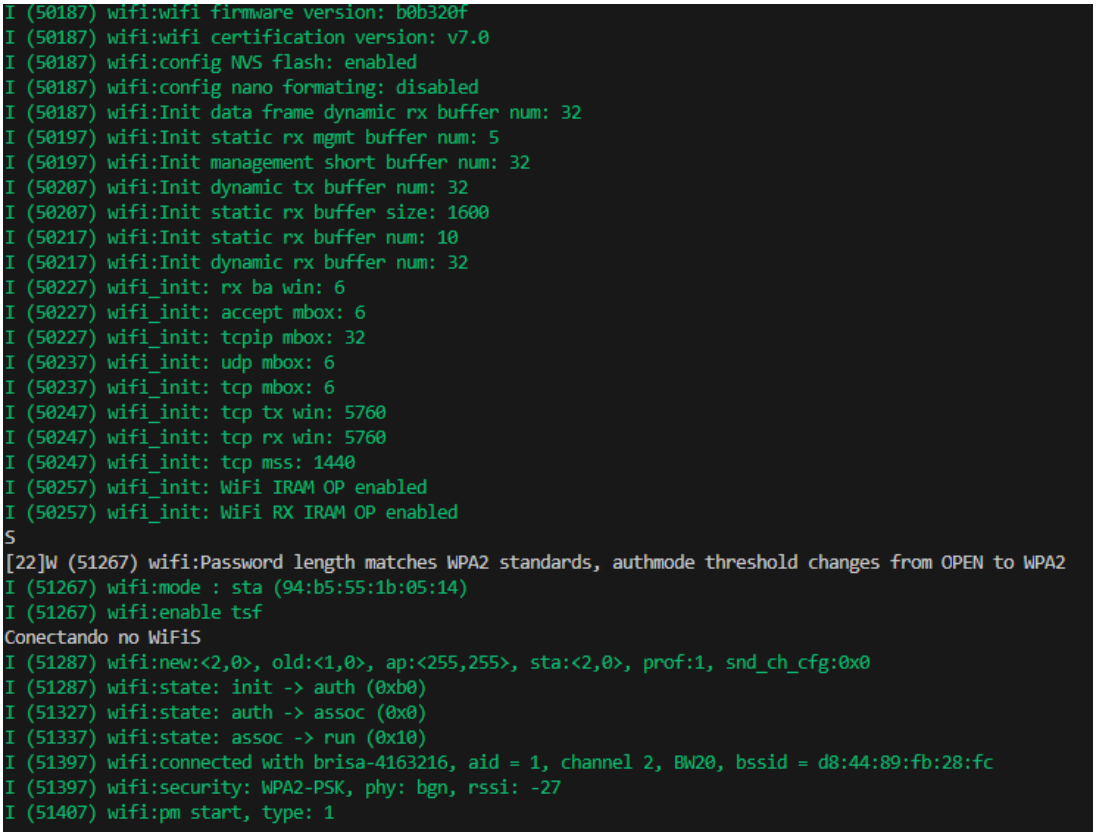


Figura 42: Terceiro *log* da aplicação *BLE + Wi-Fi*.

Fonte: Elaborado pelo autor, (2025)

A figura 42 mostra os *logs* do *framework* da *ESP32* referentes a inicialização do *Wi-Fi*.

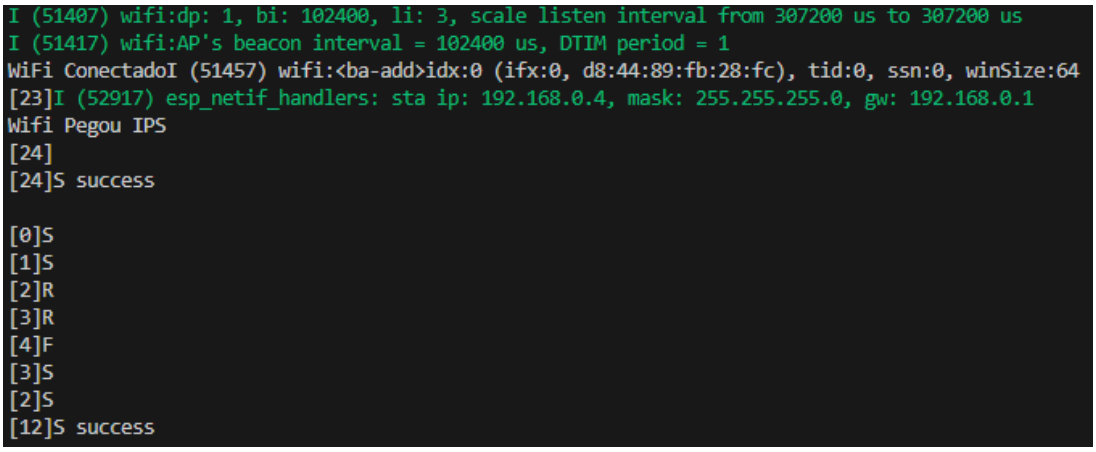


Figura 43: Quarto log da aplicação *BLE + Wi-Fi*.

Fonte: Elaborado pelo autor, (2025)

Na figura 43, mostra o final da estrutura sendo executada, onde o nó 19, que é o nó de ping, consegue obter sucesso, validando a conexão. Com isso, têm-se loop periódico da estrutura para validação.

4.9.6 Sexto caso: Teste de aplicação *bluetooth* com mensagens

Considerando um processamento simples de mensagens via *bluetooth*, onde recebemos uma mensagem para piscar um *LED* (*Light-Emitting Diode* ou Diodo Emissor de Luz), um comando de *standBy*, que espera a próxima mensagem válida a ser recebida para continuar a execução da árvore, e uma ramificação para mostrar no terminal a mensagem.

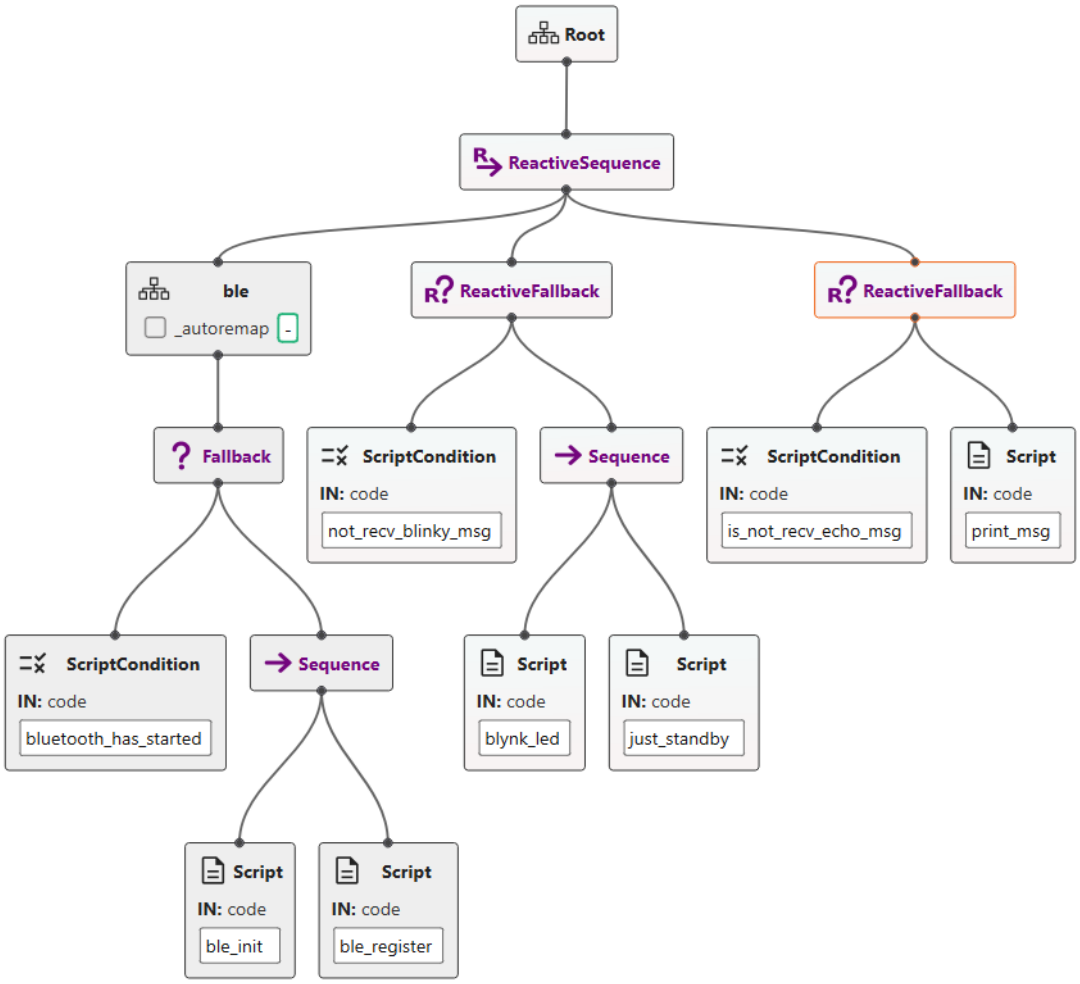


Figura 44: Árvore de comportamento para mensagem *bluetooth*.

Fonte: Elaborado pelo autor, (2025)

De acordo com a figura 44, podemos ver que as condições são reativas, enquanto as ações não. As mensagens via *bluetooth* tem dois prefixos, onde uma delas é "*led*", onde acende e apaga o *LED*, e a outra mensagem é "*print*", onde é impressa no terminal a mensagem depois do espaçamento.

Quando é enviado um comando dpara o *LED*, a aplicação entra em *standBy*, e quando é executada quando receber uma nova mensagem via *bluetooth*, irá forçar um novo processamento da mensagem, a partir do primeiro nó reativo da estrutura, que é o nó *not_recv_blinky_msg*.

```
const btree_definition_node_t btree_main_array[BTREE_MAIN_ARRAY_SIZE] =
{
        [0] = BTREE_DEFINITION_CREATE_NODE_CONDITION(3, 1, &btree_main_ble_bluetooth_has_started), \
        [1] = BTREE_DEFINITION_CREATE_NODE_ACTION(2, BTREE_DEFINITION_TREE_UNRELATED, &btree_main_ble_ble_init), \
        [2] = BTREE_DEFINITION_CREATE_NODE_ACTION(3, BTREE_DEFINITION_TREE_UNRELATED, &btree_main_ble_ble_register), \
        [3] = BTREE_DEFINITION_CREATE_NODE_REACTIVE_CONDITION(6, 4, &btree_main_main_ble_not_recv_blinky_msg), \
        [4] = BTREE_DEFINITION_CREATE_NODE_ACTION(5, BTREE_DEFINITION_TREE_UNRELATED, &btree_main_main_ble_blynk_led), \
        [5] = BTREE_DEFINITION_CREATE_NODE_ACTION(6, BTREE_DEFINITION_TREE_UNRELATED, &btree_main_main_ble_just_standby), \
        [6] = BTREE_DEFINITION_CREATE_NODE_REACTIVE_CONDITION(BTREE_DEFINITION_TREE_UNRELATED, 7, &btree_main_main_ble_is_not_recv_echo_msg), \
        [7] = BTREE_DEFINITION_CREATE_NODE_REACTIVE_ACTION(BTREE_DEFINITION_TREE_UNRELATED, BTREE_DEFINITION_TREE_UNRELATED, &btree_main_main_ble_print_msg), \
};
```

Figura 45: Estrutura mínima de comportamento para mensagem *bluetooth*.

Fonte: Elaborado pelo autor, (2025)

A figura 45 mostra a enumeração dos nós da estrutura mínima da árvore de comportamento retratada na figura 44.

```
[0]F
[1]Initializing BLEI (1547) BTDM_INIT: BT controlle
I (1547) BTDM_INIT: Bluetooth MAC: 94:b5:55:1b:05:1
I (1547) phy_init: phy_version 4860,6b7a6e5,Feb  6 2
5
[2]Registering BLE callbacksESP_GATTS_REG_EVTS
Advertising data set complete, starting advertising
[3]S [6]S success

[0]S
[3]S
[3]S [6]S success

[0]S
Device connected[3]S
[3]S [6]S success

[0]S
[3]S
[3]S [6]S success

[0]S
[3]S
[3]S [6]S success

[0]S
[3]S
[3]S [6]S success
```

Figura 46: *Log* 1 da estrutura mínima de comportamento para mensagem *bluetooth*.
Fonte: Elaborado pelo autor, (2025)

```
[0]S
MSG recebida: led[3]F
[3]F [4]S
[3]S [6]S success

[0]S
[3]S
[3]S [6]S success

[0]S
[3]S
[3]S [6]S success
```

Figura 47: *Log* 2 da estrutura mínima de comportamento para mensagem *bluetooth*.
Fonte: Elaborado pelo autor, (2025)

```
[0]S
[3]S
[3]S [6]S success

[0]S
[3]S
MSG recebida: printsjsjana[3]S [6]F
[3]S [6]F [7]jsjanaS success
```

Figura 48: *Log* 3 da estrutura mínima de comportamento para mensagem *bluetooth*.
Fonte: Elaborado pelo autor, (2025)

Note que quando temos uma interação com o *LED*, de acordo com a árvore, ela deveria entrar em *standBy* no nó *just_standby*, nó 5. Como é uma estrutura reativa, e em código, o *buffer* de mensagem deve ser limpa para evitar encher seu campo, então o nó reativo *not_recv_blinky_msg*

é novamente executado. Na execução do nó 4, é limpado o *buffer* e quando finalizado sua execução, o próximo *tick* retornará para o *not_recv_blinky_msg*, que interpretará que não há mensagem no *buffer*, o que torna o comportamento dinâmico.

Agora mantendo a mesma estrutura, mas não usando reatividade, temos a seguinte saída:

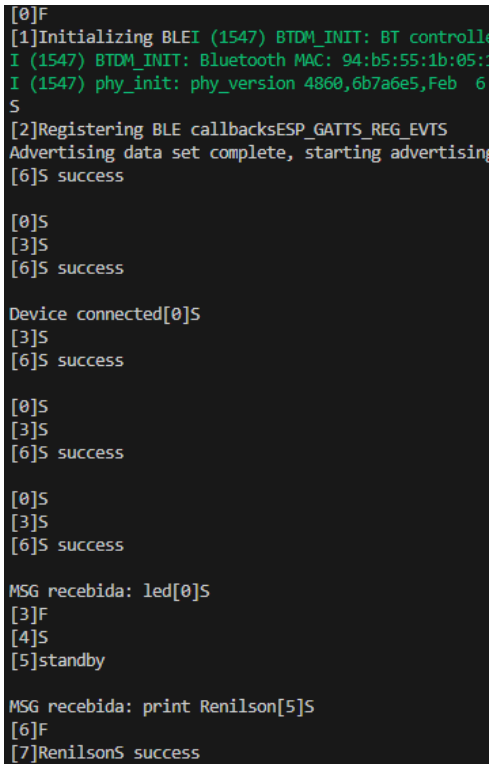


Figura 49: *Log* não reativo da estrutura mínima de comportamento para mensagem *bluetooth*.

Fonte: Elaborado pelo autor, (2025)

Note que pela figura 49 sem reatividade, executamos o nó *just_standby*, aguardando uma mensagem *bluetooth* para dar continuidade na aplicação.

5

Conclusão

Este trabalho abordou duas soluções embarcadas que podem portar uma árvore de comportamento que busca balancear o uso de memória *RAM* e *flash* de um dispositivo, mantendo o comportamento previsto de uma árvore de comportamento convencional, porém reduzido. A abordagem da estrutura sendo implementada na forma de um grafo dirigido, permite com que uma implementação embarcada mantenha as propriedades de uma árvore de comportamento de alto nível, isto é, modularidade e hierarquia, e se necessário, reatividade.

Para o funcionamento da árvore de forma otimizada, o uso de uma ferramenta para criar árvores de comportamento de forma simples e intuitiva demonstrou-se necessária, tendo uma base sólida para criar as estruturas, assim como *scripts*, dando suporte aos arquivos de saída desse ferramenta, facilitando o desenvolvimento.

O grafo de comportamento mínimo se mostrou eficiente, evitando consultas desnecessárias de lógicas que podem ser resolvidas na etapa de geração de código por meio dos *scripts*, o que permite o sistema funcionar de forma mais direta, guardando menos estruturas de nós e evitando consultas sobre o tipo de nó pai, a fim de evitar visitar o mesmo, e aplicar a sua regra.

As estruturas propostas se demonstraram viável e confiável, consultando por meio de um *buffer* de estados, que contém os estados de todos os nós do grafo de comportamento mínimo. O grafo simplificado da estrutura foi implementado no *script* `btree_execute.py` em partes, possibilitando simular a árvore convertida para um *array* compatível com a sua estrutura, e a saída desse arquivo, temos o *array* do grafo de comportamento mínimo, para determinar o comportamento de um nó em com sucesso e falha.

Os objetivos propostos no capítulo 1 foram alcançados, conseguindo executar uma árvore de comportamento de forma reduzida, o grafo de comportamento mínimo, abordado no item 3.5, assim como a reatividade foram implementadas no sistema embarcado.

A implementação da solução embarcada apresentou diversos desafios pela restrição de memória, onde cada nó deve ter um estado próprio que deve estar em *RAM*, onde apesar das estruturas estarem por padrão em *flash*, o índice no *array* de estruturas, serve como base para

saber a posição em um *buffer* que guarda os estados de todos os nós, utilizando apenas dois *bits* para cada nó, o que possibilita uma economia de espaço em *RAM*. Além disso, a implementação dos nós *decorators* com tentativas que devem guardar valores variáveis, guardam apenas um índice em um *buffer* em *RAM*, onde em *flash* é guardada apenas o endereço em que deve ser salvo. Quanto à reatividade, algo típico das árvores de comportamento, essa abordagem pode ser indesejada em sistemas embarcados se mal projetado. Uma abordagem interessante é a remoção da reatividade, o que demonstrou menos execuções para finalizar uma estrutura.

5.1 Trabalhos Futuros

- **Implementação de um *RWLock***: Implementar um *RWLock* na *blackboard* do dispositivo para evitar multiplas escritas simultâneas.
- **Otimização em nós *decorators***: Aprimorar a implementação para evitar reprocessamentos e evitar que sejam executados no final de um *tick*.
- **Otimização da reatividade**: Melhorar a abordagem da reatividade para evitar checagens constantes.
- ***Framework* próprio**: Criação de uma ferramenta própria para criação, preenchimento, manipulação, *debug* e simulação das estruturas.
- **Otimização na criação do código**: Otimizar os *scripts*, para facilitar a inclusão de novos tipos de nós.
- **Suporte a novos tipos de nós**: Inclusão de novos tipos de nós, para sistemas mais complexos.
- **Economia de memória**: Investigar outras técnicas para otimizar memória *flash* e *RAM*, lidando com *arrays* ou *buffers* externos em estruturas que se repetem.

Bibliografia

ALFREDO LIMA MOURA SILVA. *Comunicação pessoal com o autor*. 2022. Comunicação pessoal.

BROOKS, R. A. A robust layered control system for a mobile robot. *IEEE Journal of Robotics and Automation*, v. 2, n. 1, p. 14–23, March 1986.

COLLEDANCHISE, M.; ÖGREN, P. *Behavior Trees in Robotics and AI: An Introduction*. 1st. ed. Boca Raton, FL: Chapman Hall, 2018. Acesso em: 06 mai. 2024. ISBN 9781138593732. Disponível em: <https://arxiv.org/abs/1709.00084>.

DENARDIN, G. W.; BARRIQUELLO, C. H. *Sistemas Operacionais de Tempo Real e sua Aplicação em Sistemas Embarcados*. São Paulo: Blucher, 2019. ISBN 9788521213963. Disponível em: <https://storage.blucher.com.br/book/pdf_preview/9788521213963-amostra.pdf>.

FACONTI, D. *BehaviorTree.CPP: Behavior Trees in C++*. 2023. <https://github.com/BehaviorTree/BehaviorTree.CPP>. Acesso em: 15 maio 2025.

FEOFILOFF, P. Algoritmos para grafos. Apr 2020. Disponível em: <https://www.ime.usp.br/~pf/algoritmos_para_grafos/>.

Groot2. *Groot2: A graphical editor for Behavior Trees*. 2024. <https://www.behaviortree.dev/groot/>. Acessado em: 23 abr. 2025.

IOVINO, M. et al. Comparison between behavior trees and finite state machines. *arXiv preprint arXiv:2405.16137*, maio 2024.

LEVADA, A. L. M. *Teoria dos Grafos para Computação*. [S.l.: s.n.], 2022.

MICROSOFT. *Halo 2: AI Behavior List*: Documentação técnica do halo: The master chief collection. 2023. Acesso em: 28 jan. 2025. Disponível em: <https://learn.microsoft.com/en-us/halo-master-chief-collection/h2/ai/aibehaviorlist>.

NEEDHAM, M.; HODLER, A. E. *Graph Algorithms*. Sebastopol, CA: O'Reilly Media, 2019. Acesso em: 12 fev. 2025. Disponível em: <https://www.oreilly.com/library/view/graph-algorithms/9781492047674/>.

SHELBY, Z. et al. *Terminology for Constrained-Node Networks*. 2014. RFC 7228. Disponível em: <https://datatracker.ietf.org/doc/html/rfc7228>.

THOMPSON, T. History of ai in games – halflife halo. 2023. Acesso em: 27 jan. 2025. Disponível em: <https://modl.ai/halflife-halo/>.

WHITE, E. *Construindo Sistemas Embarcados: Padrões de Projeto para um Ótimo Software*. 2. ed. São Paulo: Novatec Editora, 2024. ISBN 978-85-7522-919-4. Disponível em: <https://books.google.com.br/books?id=f9YeEQAAQBAJ&printsec=frontcover&hl=pt-BR#v=onepage&q&f=false>.

Apêndice A

Repositório de Código-Fonte

O código-fonte desenvolvido como parte deste trabalho está disponível publicamente no repositório:

- **GitHub:** https://github.com/RenilsonA/Embedded-Behavior-Tree

O repositório contém o processamento de grafos de comportamento mínimo, assim como o código de exemplos citados ao longo do tabalho.